Laboratory Exercises

Instructor Notes

Laboratory 1: First NodeBuilder Project

Overview

The NodeBuilder tool is a hardware and software platform that is used to develop applications for devices with a Neuron core. The software is an LNS plug-in designed to be launched from the LonMaker tool which ships as part of the NodeBuilder platform. The LonMaker tool is a multi-purpose LONWORKS network tool that uses Visio 2003 as a graphical interface. The NodeBuilder also ships with wizards for making the process of device development an easier one.

Objectives

- Create a new LonMaker network and start NodeBuilder from LonMaker
- Create a new NodeBuilder project
- Build a simple device application

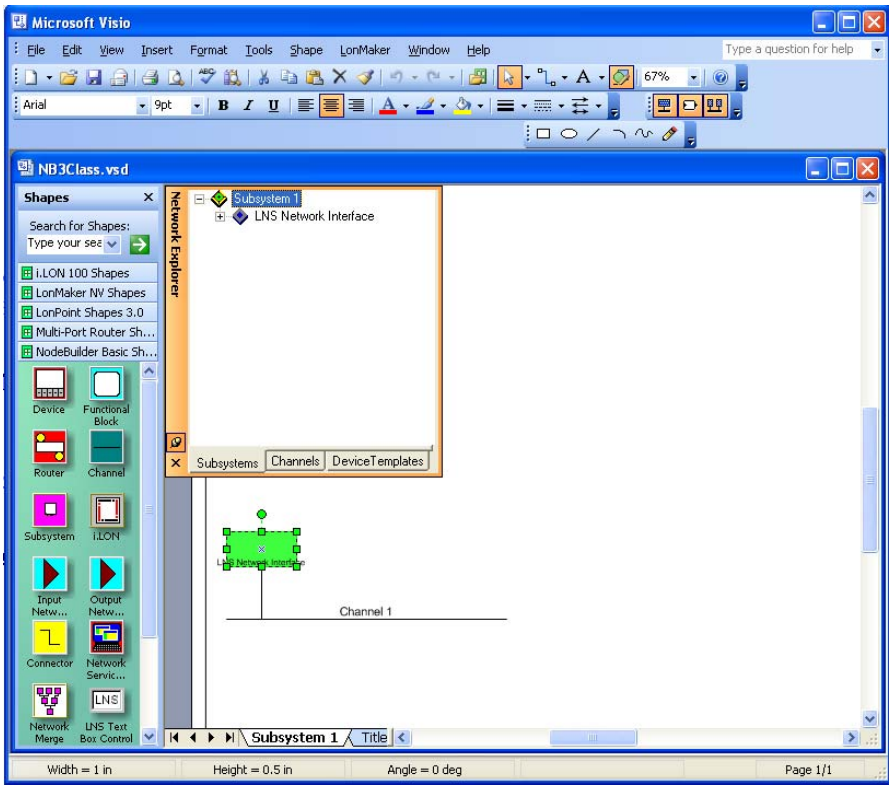
Description

In this exercise, you will create a LONWORKS device using NodeBuilder. You will create a new device template using the NodeBuilder tool, compile it, and load it to the development target device hardware. The development target device hardware in your laboratory setup is a TP/FT-10F 64K control module attached to the Gizmo 4 hardware. Although the resulting simple device will not actually do anything constructive, the objective of this lab is to walk through the steps used to create a device from scratch using the NodeBuilder development environment and get a feel for what you will be learning in the rest of this class.

Instructions

Part 1: Create a LonMaker Network

1. Double-click the LonMaker icon on the desktop or launch LonMaker from Start/Programs.
2. Type *NB3Class* into **Network name** text box.
3. Ensure that the **Show all options** is checked and click **Create Network**.
4. Click **Enable Macros** in the Security Warning which opens.
5. Click the **Next** button in the Network Wizard that opens.
6. Check **Network attached** option. Select an interface from the **Network interface name** dropdown, typically *LON1*. (Your instructor will tell you if the network interface differs on your system.)
7. Click **Next** and then select **OnNet** radio button as the **Management Mode**. Click **Next** again.
8. Select **Echelon LonMaker Browser** in the **Not Registered** list and click the **Register**. Repeat for **Echelon NodeBuilder Project Manager**.
9. Click **Finish**. A **Visio** page containing a rectangular green shape should be on your screen now as per the following screen shot.



Part 2: Start the NodeBuilder tool from the LonMaker tool and create a new project

1. From the **LonMaker** dropdown menu, select **NodeBuilder**.
2. If NodeBuilder has not been used before, you will need to create a new project. Select **Create a new NodeBuilder project** and click **Next**.
3. Use the default **Project name,** which is "NB3Class" (the same name as the network drawing), and click **Next**.
4. Add **$LonWorks$\NodeBuilder\Gizmo4** to the **Include Search Path** text field.
5. Click **Finish** to run the **NodeBuilder New Device Template Wizard**.

Part 3: Create a new NodeBuilder Device Template

A NodeBuilder Device template contains references to the application source files and to the hardware template, which is a file containing a description of the device hardware platform.

1. Enter *Simple1* as the **NodeBuilder device template name**. NodeBuilder will use the source file name ***Simple1.nc*** automatically. (Note for future reference the folder where NodeBuilder will create and maintain the files related to the new device template.)
2. Click **Next** and the **Program ID** dialog box opens.
3. Click the **Calculator** button to launch the **LONMARK Standard Program ID Calculator**. Fill in the calculator with the following information, using the defaults for all other information not specified.

Manufacturer	Examples
Category	I/O
Device class	Multi I/O module
Usage	Industrial/commercial
Channel type	TP/FT-10

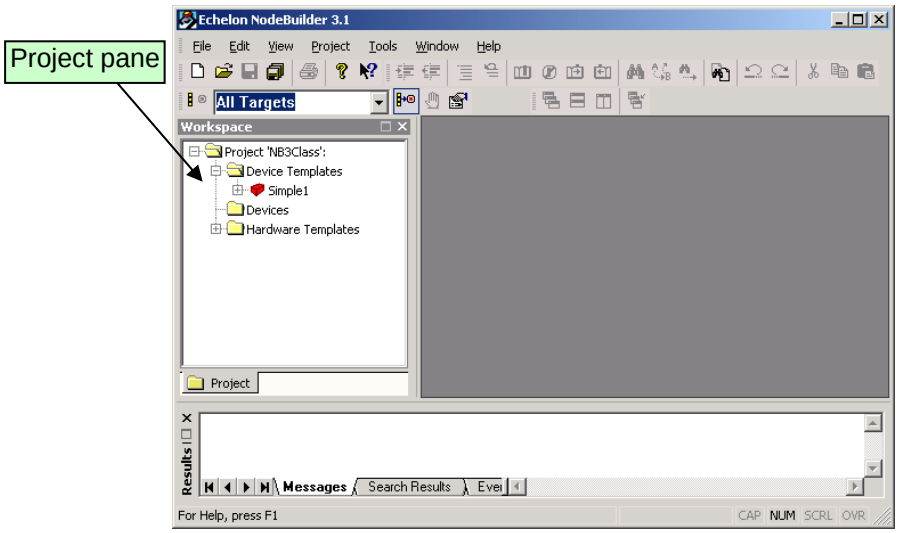
4. Click **OK** to return to the **Program ID** dialog box, and click **Next**.
5. Choose *TPFT-10F 64K Control Module* in the **Development build hardware template** drop-down menu. **Do not** enter anything in the **Release build hardware template** dropdown menu.

For the purposes of the labs in this course we will be using a generic I/O board called the Gizmo 4 which can be either connected to a NodeBuilder node (also known as the "LTM-10A module" used for development) or have a control module plugged into it. A control module consists of a Neuron Chip, a transceiver, and some memory (in this case the transceiver is an FTT-10A free topology transceiver and the memory used is a 64Kbytes external FLASH).

6. *Uncheck* the **Run NodeBuilder Code Wizard** checkbox and click the **Finish** button.

Part 4: Build a simple application

1. In the *Project pane* (titled *Workspace*), which displays the files and templates defined for your project, expand the ***Device Templates*** tree to find the main *.nc* file for the *Simple1* device template. In our case the file is called **Simple1.nc** and will be in red, which indicates the file is missing.



2. Double-click on the ***Simple1.nc*** file and answer **Yes** to the question asking if you wish to create a new file.
3. Fill in the empty file with the following Neuron C code (you can use Copy/Paste):

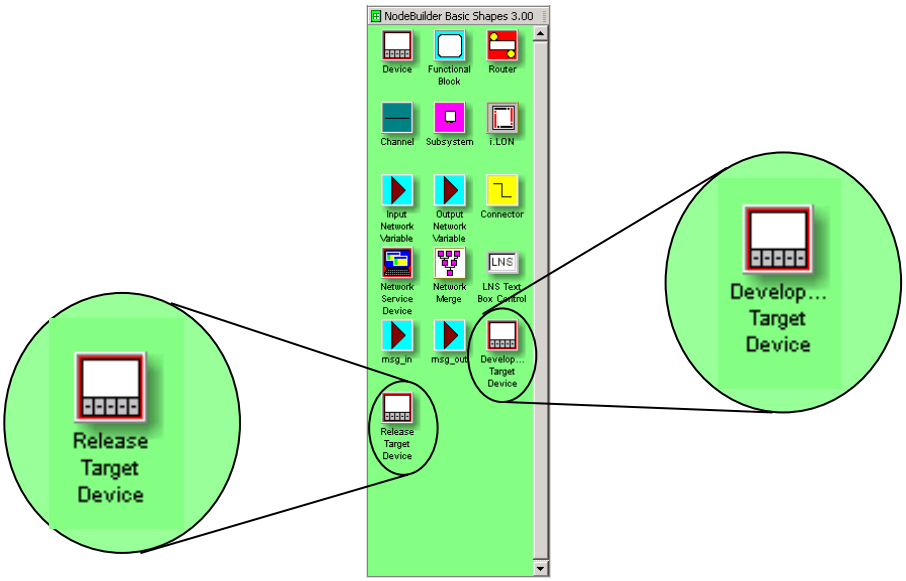
```
#pragma num_alias_table_entries 15
when (reset)
{
//This device does nothing, just compiles
}
```

4. In the *Project pane*, right click on the icon for the ***Simple1*** device template and select **Build** from the pop-up menu.

NodeBuilder will compile, link and export the device template into the LonMaker network. The output of the build is called the "device files set" and it includes the APB, XIF and NEI files. We will discuss these files later in this class.

Part 5: Add a Development Target Device to the LonMaker network

The NodeBuilder 3 Basic shapes stencil in the LonMaker tool contains two device shapes for use during development. The ***Development Target Device*** *shape refers to the development hardware and the* ***Release Target Device*** *shape refers to your release hardware. For all the laboratories in this class, we will use the TP/FT-10F control module with the Gizmo 4 hardware. (Some classrooms may use the NodeBuilder LTM-10A platform with the Gizmo 4 hardware.)*



5. Switch to the **LonMaker** application (the **Visio** project opened in the background).
6. From the **NodeBuilder Basic Shapes** stencil, drag and drop a **Development Target Device** shape onto the drawing.
7. Accept the default name *Device 1* for the new device.
8. Check the **Commission Device** checkbox.
9. Select *Simple1* in the **NodeBuilder Device Template** drop-down and click **Next**.
10. Accept the defaults and click **Next** until **Specify device application image name** appears at the top of the dialog box. On this screen check the **Load Application Image** and **Update firmware in device to match application image** checkboxes. Click **Next**.
11. Select **Online** for the state of the device and click **Finish**.
12. When prompted to **Please press the service pin on the device**, press the SERVICE button on the Gizmo 4.

*You might get a warning here if there's already a program with a different ID in the Gizmo device. This is ok, just click **Yes** to continue and overwrite the current content.*

What is the color of the target device shape?

13. Right click on the device shape you just created, select **Manage**, and click the **Test** button.

Read the test results and note if there are any errors.

14. Close the **LonMaker Device Manager** window.
15. Drag the blue **Functional Block** shape from the **NodeBuilder Basic Shapes** stencil onto the drawing.
16. Select *Device 1* from the **Name** dropdown box in the **Device** frame on the **Functional Block Wizard** dialog box.
17. Note the available items in the **Functional Block** dropdown box.

Why are there so few?

*The **Virtual Functional Block** is LonMaker's representation of a device's interface at the device scope and is available for every device. The network variable and configuration properties exposed through this functional block are actually not members of any functional block and pertain to the device as a whole.*

18. Click on **Cancel** to return to the LonMaker drawing without adding the functional block.

You have just finished your first LONWORKS device development project. You will add functional blocks, network variables, and configuration properties to this device in the next laboratory exercise.

Laboratory 2: Adding a Device Interface to a Device

Overview

Applications in devices are divided into one or more functional blocks. A functional block is an implementation of a functional profile with a collection of network variables and configuration properties. A functional profile defines mandatory and optional network variables and configuration properties for a type of functional block. A device interface includes all functional blocks, network variables, configuration properties and configuration property default values defined in the application. When an application is built, NodeBuilder automatically produces a text-based device interface file with a .XIF extension.

Objectives

- Add a functional block to a device
- Add network variables and a configuration property to a device
- Build and load the new application
- Use the LonMaker Browser to verify the device interface

Description

In this laboratory, you will define a functional block for the device you created in the previous lab. You will add a LONMARK Open Loop Sensor functional block (Object Type 1) to the device and a few network variables and configuration properties. You will use the LonMaker tool to verify the functional blocks, network variables and configuration properties in your device.

Instructions

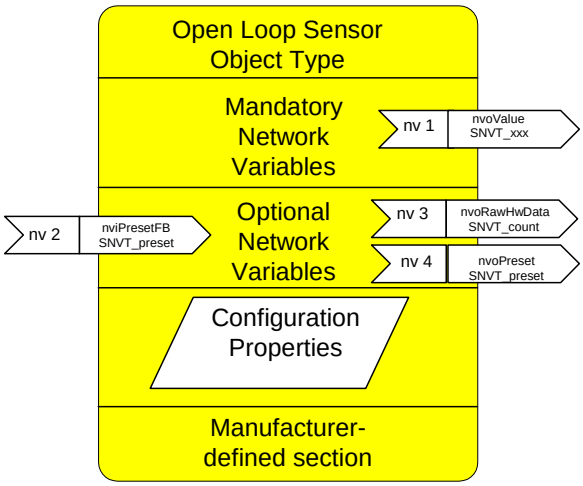
Part 1: Add a functional block and device interface to the device

1. If not still running from the previous lab, open the LonMaker network from the previous lab and launch the NodeBuilder development tool.
2. In NodeBuilder, double-click on the *Simple1* device template, click on the **Calculator** button under the **Program ID** tab and check that the program ID settings match those in the table below. Click **OK** when you are done to return to the **NodeBuilder Device Template Properties** dialog box.

Manufacturer	Examples
Category	I/O
Device Class	Multi I/O module
Usage	Industrial/commercial
Channel Type	TP/FT-10

3. Ensure that **Enable** the **Automatic program ID management** is enabled. Set the **Min model #** to *0x00* and **Max model #** to *0x07* and click **OK**.

Next you will use NodeBuilder's editor to add an Open Loop Sensor functional block to the Simple1.nc file. Below is the graphical representation of the object.



Below is the specification of the device interface you will add to your device.

Network Variable Name	Direction	Type	Specification
nvoValue	Output	SNVT_temp	Mandatory
Myfirstnv	Input	SNVT_switch	User specific
mysecondnv	Output	SNVT_switch	User specific

Configuration NV	Applies To	Default Value
SCPTMaxSendTime	mysecondnv Network Variable	50

The SCPTMaxSendTime configuration property is used to control the maximum period of time that can pass before the functional block automatically transmits the current value of the related network variable. This is also known as the heartbeat function which we will review later in the class. SCPTMaxSendTime has a resolution of 0.1s.

Configuration property values are stored in both the device and the LNS database with the objective of simplifying subsequent device replacement in the event that the device breaks.

4. At the top of the *Simple1.nc* file, insert the following code after the line `#pragma num_alias_table_entries 15` and before `when (reset)`.

```
// Fblock CP Family Declarations
network input cp SCPTmaxSendTime nciMaxSendTime;
// Fblock Input NV Declarations
network input SNVT_switch myfirstnv;

// Fblock Output NV Declarations
network output SNVT_temp mandnv;
network output SNVT_switch mysecondnv
nv_properties {
   nciMaxSendTime=500
};

// Fblock Declaration
fblock SFPTopenLoopSensor {
   mandnv      implements nvoValue;
   myfirstnv   implementation_specific (513) nviImpSwitch;
   mysecondnv  implementation_specific (514) nvoImpSwitch;
} openLoopSensor;
```

In your own words, what does the fblock keyword do?

How is nciMaxSendTime associated with the network variable mysecondnv?

Part 2: Build the new image and load it to the target device

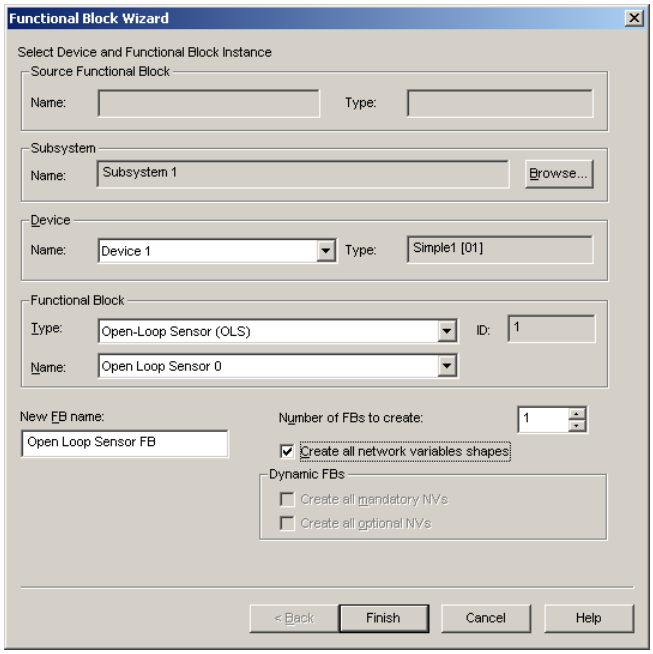
The NodeBuilder Device template contains references to the application source files and the hardware template which is a file containing a description of the device hardware platform.

1. Right-click on the *Simple1* device template icon in the project tree and select **Build**.
2. If you receive any build errors, double-check the code you entered.
3. If you were still using the LonMaker network used in Lab 1, the **Application Image Wizard** should have been automatically launched. In this case go to step 10, otherwise continue with step 4.
4. Switch to the LonMaker application, right-click on the device you previously added to the network and select **Delete**.
5. From within the Network Explorer (expand if necessary) select the Device Templates tab, right click the *Simple1* device template from the tree list and select delete.
6. Drag a new **Development Target Device** shape onto the network from the **NodeBuilder Basic Shapes 3** stencil.
7. Name the new device *Device 1*, check the **Commission Device** checkbox and click **Next**.
8. Use the newly created LNS Device Template, Simple1[x], with this device. [x] depends on the number of builds you performed and is also the version number of the device.
9. Accept the defaults and click **Next** until you are asked whether or not to load the device. Check the **Load Application Image** checkbox and click **Next**.
10. Select **Online** for the state of the device and click **Finish**.
11. If you executed steps 4-9 then you will be prompted to press the service pin on the target device; press the service button on the Gizmo 4.

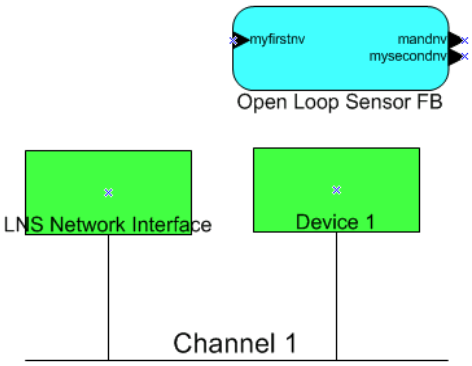
In the list of Device Templates (accessible from the Network Explorer Device Template tab) registered with the LonMaker network you will find the device template exported by NodeBuilder. The Device Template represents objects present in the LonMaker database after NodeBuilder exports an XIF file. Like an XIF file, every Device Template must have a unique Program ID. When Automatic Program ID Management is enabled, NodeBuilder will increment the last nibble of the Program ID and export the device's XIF file into a new device template. Without this feature or changing the Program ID manually you would be forced to first delete an existing Device Template and rebuild the device using NodeBuilder before the interface changes would take effect in LonMaker.

Part 3: Add the functional block to the drawing

1. Drag a **Functional Block** shape from the **NodeBuilder Basic Shapes** stencil onto the drawing.
2. Select *Device 1* in the **Device Name** drop-down list and *Open Loop Sensor 0* in the **Functional Block Name** drop-down list. If the *Open Loop Sensor 0* functional block is not available, go back and check the code you entered, rebuild the device template, and repeat the process.
3. Give the functional block a meaningful name such as *Open Loop Sensor FB*. Check the **Create shapes for all network variables** checkbox and click **Finish**.



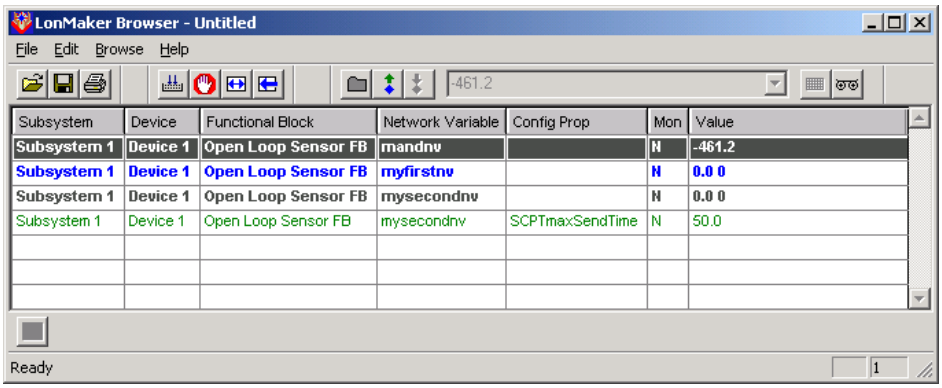
Below is an example of what your drawing should look like after you finish this process.



Part 4: Browse the functional block

The LonMaker Browser allows you to view and modify network variables and configuration properties on one or more functional blocks.

1. Right-click the functional block shape and select **Browse** from the pop-up menu. Use the LonMaker Browser to confirm that the **Open Loop Sensor FB** has the correct configuration properties and network variables.



2. If the network variables and configuration properties do not match the specifications, go back and check the code you entered, recompile the template, and repeat the process.
3. Close the Browser when you are done.

Congratulations. You have just finished adding a functional block and the device interface to your device... but wouldn't it be nice if there were a software wizard to generate the code for you?

Laboratory 3: Using the NodeBuilder Code Wizard

Overview

The NodeBuilder Code Wizard has been developed to simplify and speed the development of LONMARK based devices. It allows the developer to visually create LONMARK compliant Device Interfaces for Neuron Chip and Smart Transceiver based devices, thus allowing focus on the development of features that differentiate his product from those of competitors.

Objectives

- Create a new NodeBuilder Device Template
- Create a device interface using the NodeBuilder Code Wizard
- Identify which Wizard-generated files pertain to the device as a whole, and which pertain to individual functional blocks
- Build and load the device application
- Create new device and functional block shapes in the LonMaker tool

Description

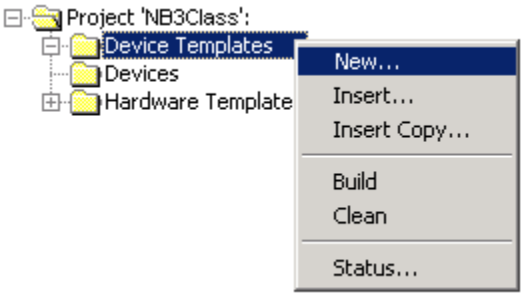
You will review the steps involved in the creation of a new NodeBuilder Device Template, including using the **Program ID Calculator** and selecting the **Development** and **Release Hardware Templates**. You will use the NodeBuilder Code Wizard to create the device interface for a LONMARK-compliant device containing two Standard Functional Profile Templates (SFPTs): a Light Sensor SFPT and an Occupancy sensor SFP. You will examine the file structure and contents of the files created by the NodeBuilder Code Wizard. You will build and load the Wizard-generated device application into the Gizmo 4/TPFT-10F 64K control module development hardware. Finally, you will use the LonMaker tool to create a new device and functional block shapes representing the Light and Occupancy sensor device.

This laboratory will demonstrate how easy it is for a developer to create a well-behaved LONMARK-compliant device using the NodeBuilder Code Wizard.

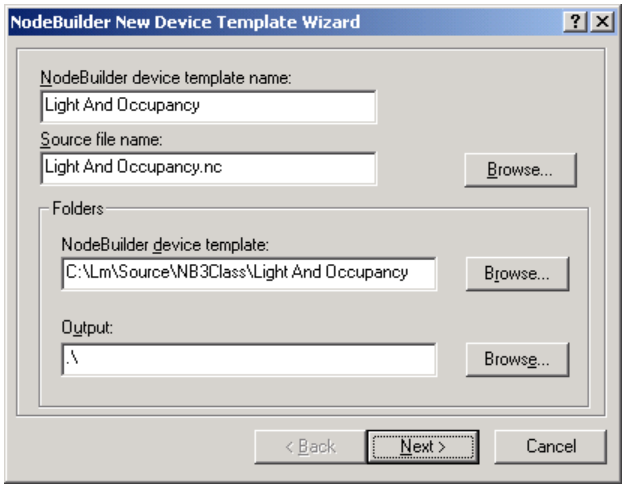
Instructions

Part 1: Create a new NodeBuilder Device Template

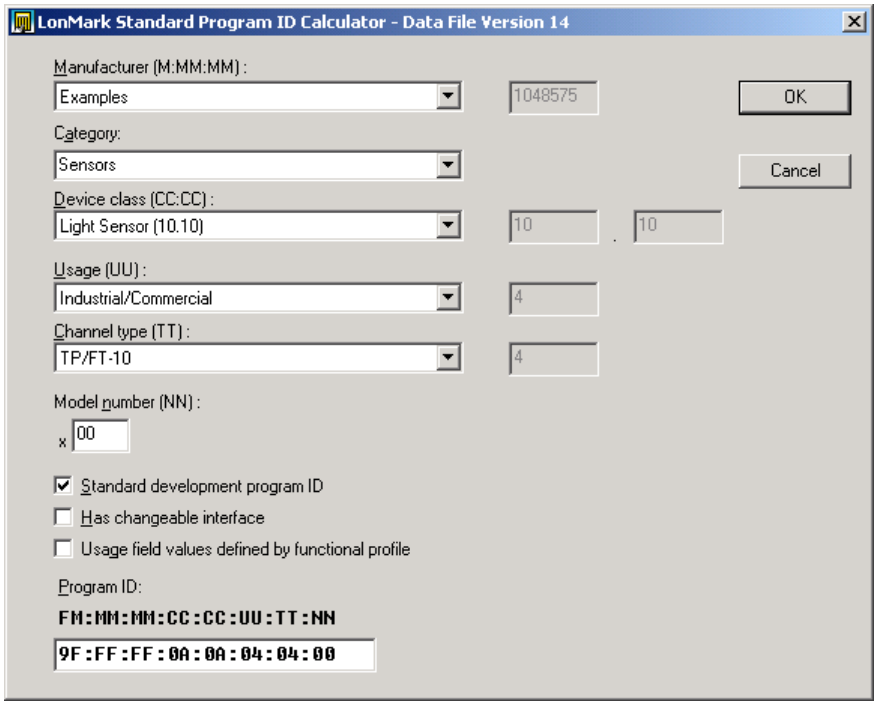
1. If not still running from the previous lab, open the LonMaker network from the previous lab and launch the *NodeBuilder* development tool.
2. In the project pane, right click on the *Device Templates* folder and select **New…** from the popup menu.



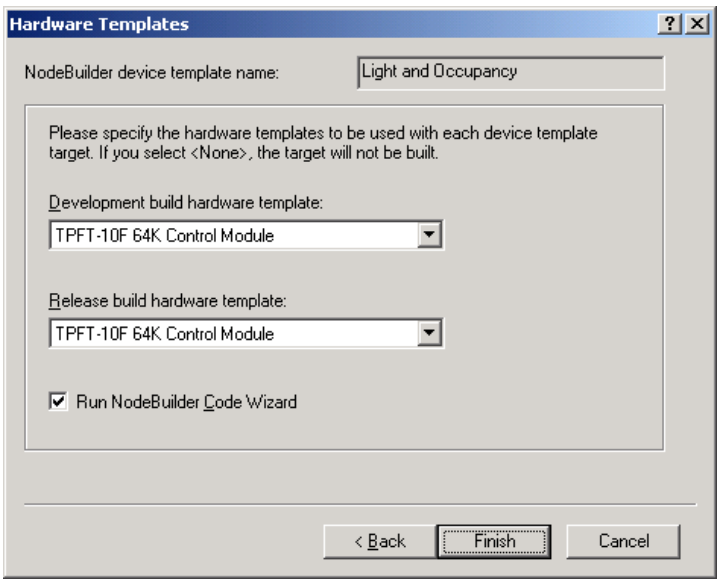
3. Specify the **NodeBuilder device template name** as *Light and Occupancy*, and accept the automatically filled-in **Source file name** *Light and Occupancy.nc* as shown below. Click **Next**.



4. Use the **LonMark Standard Program ID Calculator** to set the Program ID as shown below and then click **OK.**



5. Click **Next**. Choose *TPFT-10F 64K Control Module* as the hardware template for both the **Development build hardware template** and **Release build hardware template**. Check the **Run NodeBuilder Code Wizard** checkbox and click **Finish**. The **NodeBuilder Code Wizard** will open.

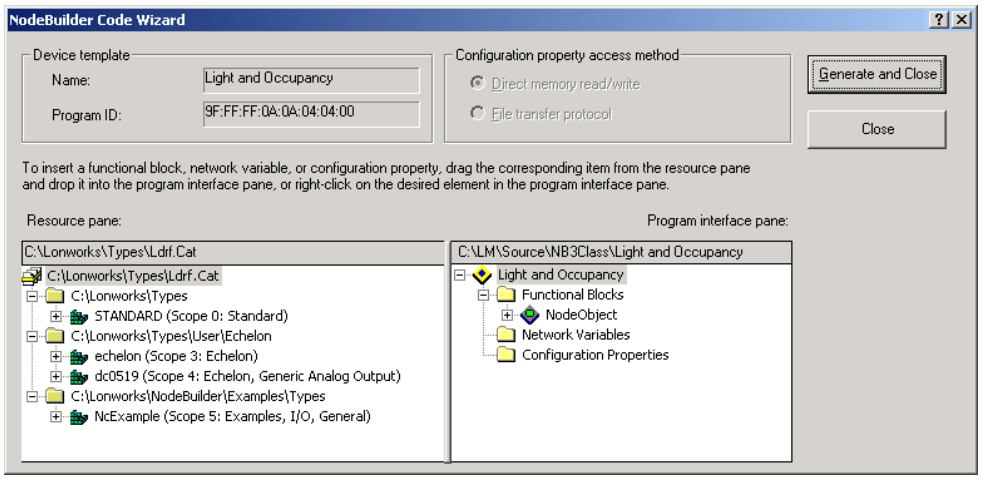


Part 2: Use the NodeBuilder Code Wizard to create the device interface

In Laboratory 2, you manually created network variables and functional blocks with Neuron C code. In this laboratory you will use the NodeBuilder Code Wizard to generate the device interface for our Light and Occupancy Sensor.

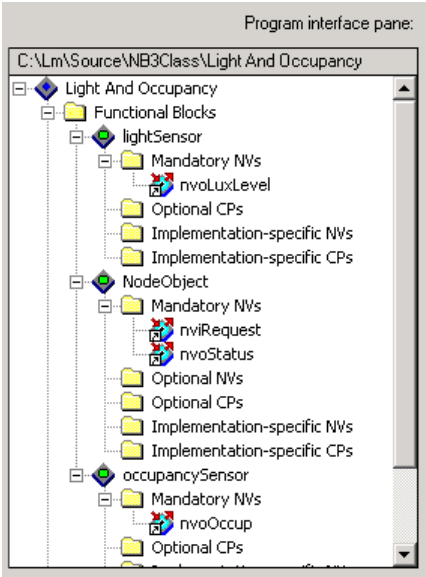
The Code Wizard will automatically create the Neuron C code and logically organize it by means of a file structure that will match with the functional blocks used in our device.

1. The **NodeBuilder Code Wizard** window should be open on your screen from the previous procedure.



2. In the **Resource pane**, expand the *Standard (Scope 0: Standard)* branch and then the *Functional Profile Templates* branch and look for the *SFPTlightSensor (1010)* functional profile. Drag and drop the profile into the **Program interface pane**. Repeat this process with the *SFPToccupancySensor (1060)* functional profile.

3. Expand all the branches in the **Program interface pane** to see that the three functional blocks of your device contain only the mandatory network variables as specified by the *LONMARK Interoperability Guidelines*. The following picture shows what you should have in your screen.



Note that the Node Object is created by default since a LONMARK-compliant device with more than one functional block must always contain a node object.

4. From your understanding of the LONMARK Interoperability Guidelines answer the following:

   What is the difference between Mandatory and Optional NVs?

   _____

   _____

   What is the difference between Optional and Implementation-specific NVs?

   _____

   _____

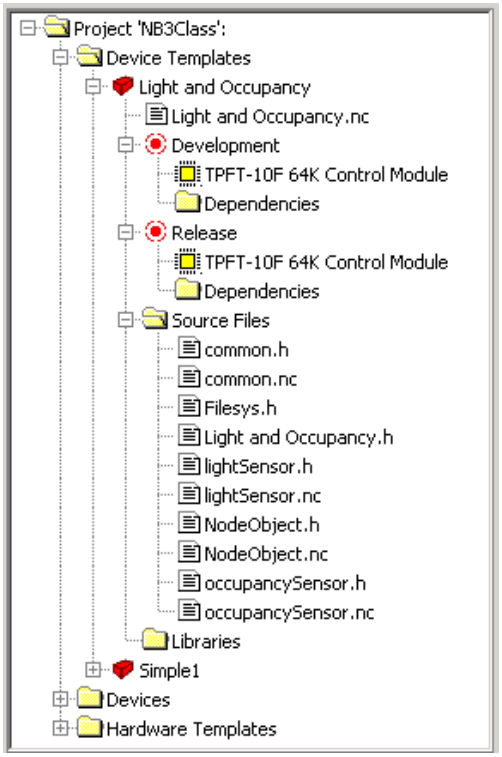
   Are the same differences applicable for configuration properties (CPs)?

   _____

   _____

5. Click **Generate and Close** and the Code Wizard will generate the Neuron C source files.

6. In the **Project pane**, expand all the branches under your newly created *Light and Occupancy* device template. The tree should look like the one in the picture below.



7. Using any of your course materials and documentation, briefly answer the following questions:

   What information is stored in the *lightSensor.*, occupancySensor.* and NodeObject.** source files?

   _____

   _____

   What are the *common.** and *Filesys.h* source files for?

   _____

   _____

   What events are handled in the *Light and Occupancy.nc* source file?

   _____

   _____

   Do those events apply to a particular functional block or to the device as a whole? _____

   To which file would you add an include statement for a header useful to all of the functional blocks on the device? What line would you place it on?

   _____

   _____

   If you were to declare a global variable necessary for calculations by the Light Sensor functional block in which file and on which line would you place the declaration?

   _____

   _____

   If you were to declare an I/O driver to track occupancy from a motion sensor in which file and on which line would you place the declaration? Where would you place the code to handle the change of state for the driver?

   _____

   _____

   If you were to handle an event conveying a change of luminescence in the room, which file and on which line would you place the code?

   _____

   _____

8. Following is a piece of code extracted from the *lightSensor.h* source file. Use the line numbers to answer the following questions.

```
1   //{{NodeBuilder Code Wizard Start
2   // The NodeBuilder Code Wizard will add and remove code here.
3   // DO NOT EDIT the NodeBuilder Code Wizard generated code in
4   // these blocks between {{NodeBuilder Code Wizard Start
5   // and }}NodeBuilder Code Wizard End
6   //<Include Enum Type Headers>
7   //
8   //<Fblock Global Index>
9   #define FBLOCK_lightSensor_INDEX    lightSensor::global_index
10  //
11  //<Forward FBDirector>
12  extern void lightSensorDirector(unsigned uFblockIndex,
                                    int nCommand);
13  //<Forward NVProcessor>
14  //
15  //<Fblock CP Family Declarations>
16  //
17  //<Fblock Input NV Declarations>
18  //
19  //<Fblock Output NV Declarations>
20  network output SNVT_lux nvoLuxLevel;
21  //
22  //<Fblock Declaration>
23  fblock SFPTlightSensor {
24        nvoLuxLevel  implements nvoLuxLevel;
25        director     lightSensorDirector;
26  } lightSensor
27  #ifdef USE_EXTERNAL_NAME
28  external_name("lightSensor")
29  #endif
30  ;
31  //
32  //}}NodeBuilder Code Wizard End
```

What is declared in line 23, and what is its relationship with line 26?

_____

_____

What is declared in line 20, and what is its relationship with line 24?

_____

_____

9. Build the project by doing a right click on the *Light and Occupancy* device template and selecting **Build**. Check the Results pane at the bottom of the window.

   Are there any errors reported? _____

   Are there any warnings? _____

   In your opinion, why did you get those warnings (if any)?

   _____

   _____

   How many times is the XIF file imported into the LonMaker database considering the fact that NodeBuilder generated one file set for the Development target and a separate file set for the Release target?

   _____

   Why?

   _____

   _____

Part 3: Add the functional block to the drawing

Just as in Laboratory 2, you can use the LonMaker tool to drop a new Development Target device shape that uses the Light and Occupancy LNS Device Template and drop functional block shapes for each of the functional blocks within the device.

1. Switch to the LonMaker application.
2. Right-click on the existing device shapes (there should be one named *Device 1* from the previous labs) and choose **Decommission**. Click **OK** when you are asked to confirm this command.
3. Using the process you learned in Laboratory 2, add a *Light and Occupancy* device to your network (you can name it *Light and Occupancy* as well) and add functional block shapes for each of the functional blocks you created in this laboratory. Use intuitive names such as *Light Sensor* and *Occupancy Sensor* for the functional blocks.

If your device had ten functional blocks with four network variables each, would you consider coding the device interface by hand?

Laboratory 4: Input/Output Hardware Processing

Overview

The Gizmo 4 Multi I/O board provides several useful tools for LONWORKS device developers as they create code to support their device hardware or to use common I/O devices such as an LED or a piezo buzzer. In this laboratory you will use the Gizmo 4 library to process input signals from a push button and a quadrature shaft encoder.

The Neuron Chip firmware has several built-in I/O libraries that allow you to process various types of digital I/O signals connected to the Neuron Chip without the need for extra hardware or driver software. You use these libraries in your application by declaring I/O objects. The declarations include the pin number on the Neuron Chip, the type of I/O object, the direction of the object (input or output), a name for the I/O object and, for some objects, a set of parameters to further customize the object's behavior. To access the object, you can then use the built-in event processing mechanisms in Neuron C, such as `when(io_changes(my_object))`*, to trigger events based on changes in the value of* `my_object`*, as well as using the* `io_in()` *and* `io_out()` *built-in functions to read and write to the object. The Gizmo 4 ships with NodeBuilder and is supplied with a header file with all the relevant I/O declarations and other example source code to simplify the development process.*

Objectives

- Include libraries to enable interaction with the I/O systems on the Gizmo 4 platform
- Add code to process hardware I/O
- Use the NodeBuilder debugger to catch I/O events, watch internal variable values during the execution of the application, and analyze the internal behavior of the application
- Use the LonMaker Browser to verify device operation by monitoring network variable values as hardware inputs change

Description

In this laboratory you will add hardware inputs to the *Light and Occupancy* Sensor functional blocks that were defined in the previous laboratory. To simulate a change in light level, a turn of the dial on the Gizmo 4 board will cause a change in the quadrature shaft encoder which, in turn, will trigger the *Light Sensor* functional block to update its output network variable. A change in the push button on the Gizmo 4 board will simulate a change in occupancy state for the *Occupancy Sensor* and cause this functional block to update its network variable. Once you have added code to support the hardware, you will use the NodeBuilder Debugger and the LonMaker Browser to test the functionality of the sensors. As an optional exercise, you may choose to write code to drive the Gizmo 4 display and piezo buzzer to signal changes in occupancy or light levels at the device.

Warning

This laboratory does not step you through adding code as in the previous laboratory exercises. You will be required to think and figure out the steps involved on your own.

Instructions

Part 1: Include the Gizmo 4 Library

1. If not still running from the previous lab, open the LonMaker network from Lab 3 and launch the NodeBuilder development tool. We will use the *Light and Occupancy* device defined in Lab 3.
2. Add the following #include statement in **common.h** to include the Gizmo4 library.

   ```
   #include "gizmo4.h"
   ```

3. The Gizmo 4 library is not part of the standard libraries installed by NodeBuilder, so you must add the include path to your project. To do this, go to the **Project** menu, select **Settings**..., click on the **Project** tab, and enter "$LonWorks$\NodeBuilder\Gizmo4" (without the quotes) into the Include search path field.

   You may find the path already present if you successfully specified it in Lab 1 when creating a project. Remember, the include path applies to all device templates within a single project.

Part 2: Find the object names for the I/O you will use

1. Add gizmo4.h to the list of your source files by right clicking on the Source Files folder in the Project Pane window titled **Workspace** and selecting **Insert**. gizmo4.h is located under *LonWorks\NodeBuilder\Gizmo4*.
2. Find the quadrature object name by opening the *gizmo4.h* file in the NodeBuilder editor and finding the I/O declaration for the quadrature object connected to pin I/O 4, remember that IO declarations start with IO_X where X is the pin number. Record the name of the object used in the *gizmo4.h* file so that you can access this object in your application code.

   Quadrature I/O Object name: _____

ANSWER: ioWheel

3. Find the push button object name by finding the declaration (within the same *gizmo4.h* file) of the input push button connected to pin I/O 6. Record this object name so that you can use it later in the occupancy sensor code.

   Push Button I/O Object name: _____

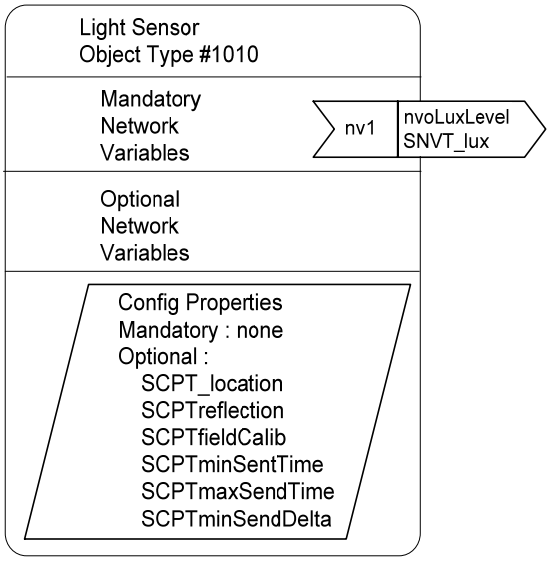
ANSWER: ioButton1

Part 3: Add Light Sensor I/O Processing

Since the Gizmo 4 hardware does not contain a real light sensor, you will simulate the intensity of the light level by turning the shaft encoder on the Gizmo *4. This shaft encoder can be read by using the quadrature I/O object you found in the previous section. Whenever a change is sensed to the quadrature object, the new lux level should be written to the nvoLuxLevel network variable. For complete documentation (including related events and the way it operates) on the quadrature I/O object, you may consult the Neuron C Reference Guide.*

- **Hardware specification**

The following describes the device interface and hardware interface for the Light Sensor functional block. The values for the Resolution and Range of the network variable come from the 1010 Light Sensor Functional Profile.



For this exercise it is sufficient to set a maximum of 1000 lux for your sensor.

Device Interface

Network variable	nvoLuxLevel
Type	SNVT_lux
Resolution	1 lux
Range	0..1000 lux

Hardware Interface

Object name	<from earlier in this laboratory>
I/O object	input quadrature

Increment speed 5 lux per unit change

- **Implementation hints**

To process the quadrature input, you will need to use a ***when(io_update_occurs(…))*** *clause. Within this when clause, you should update the* ***lightSensor::nvoLuxLevel*** *value by adding or subtracting to it the new value received on the quadrature I/O object. Remember that the range of the* ***lightSensor::nvoLuxLevel*** *network variable value goes from 0 to 1000 lux.*

Neuron C features a ***signed long*** *built-in variable called* ***input_value****, which contains the current value of the I/O model associated with the when clause event. The scope of input_value is the critical section of a given when clauses. This means you do not have to explicitly read I/O again within the critical section. The Neuron C Reference Guide has a detailed explanation on how this built-in variable works.*

Neuron C has the built in functions ***min()*** *and* ***max()*** *which can be used to provide simple variable clipping.*

You can find the definition of SNVT_Lux in the ***LonMark SNVT and SCPT Guide*** *file located in the* ***Echelon NodeBuilder Software*** *program group.*

Place variable declarations and constant definitions specific to this functional block at the end of the *lightSensor.h* file.

Add code to the end of *lightSensor.nc* file to process the quadrature input change. Be sure to use the following condition tracked the stated of the function block using fblockNormalNotLockedOut as documented in the Extra Notes for this lab through out your code. This will help to prevent unwanted execution while the functional block is disabled or locked out.

```
if (fblockNormalNotLockedOut(FBLOCK_lightSensor_INDEX)) {…}
```

1. Add code, if necessary, to the **lightSensorDirector(..)** function in the *lightSensor.nc* file to define behavior during various states of the functional block. (e.g. when the device is being *Reset*, *Enabled*, *Disabled*, etc)

An output network variable is propagated implicitly at the end of a when task. Multiple assignments to the output network variable will not cause additional traffic unless you explicitly call the ***propogate()*** *function.*

Part 4: Add Occupancy Sensor I/O Processing

Since the Gizmo 4 hardware does not contain an occupancy sensor, you will simulate movement in front of an occupancy sensor by pushing a button on the Gizmo 4. When occupancy is first detected, the network variable ***nvoOccup*** *should send a value of* ***OC_OCCUPIED****. The data type of* ***nvoOccup*** *is* ***SNVT_occupany*** *(a pre-defined enumeration).* ***nvoOccup*** *should be set back to* ***OC_UNOCCUPIED*** *five seconds after the push button is released. You may achieve this by using an application* ***timer****, which is started inside the event that gets triggered when the push button is released. Any activity on the push button within the five-second window should reset the timer.*

- **Hardware Specification**

The following describes the device interface and hardware interface for the Occupancy Sensor functional block. For details of the network variable output behavior, you may examine the 1060 Occupancy Sensor Functional Profile. In this lab exercise you will not need to implement heartbeats or throttle.

Device Interface

Network variable	nvoOccup
Type	SNVT_Occupancy
Range	range of **occup_t** Enumeration specified below

Value	Identifier	Notes
0	OC_OCCUPIED	Area is occupied
1	OC_UNOCCUPIED	Area is unoccupied
2	OC_BYPASS	Area is temporarily occupied for the bypass period
3	OC_STANDBY	Area is temporarily unoccupied
0xFF	OC_NUL	Value not available

Hardware Interface

Object name	<from earlier in this laboratory>
I/O object	input bit
Debounce	5 seconds

- **Implementation hints**

  To process the push button input, you will need to use a ***when(io_changes(…))*** *clause. Within this when clause, you should update* ***occupancySensor::nvoOccup*** *value to indicate that the sensor has detected occupancy. When the sensor ceases to detect occupancy, a timer should be started so that when it expires the* ***occupancySensor::nvoOccup*** *value gets updated accordingly. Take into account that right after the sensor ceases to detect occupancy, there could be a new person entering the room, if this occurs then the timer should be stopped.*
  It is also important to note that the initial value of ***occupancySensor::nvoOccup*** *is 0 (****OC_OCCUPIED****). Make sure it is changed at declaration time to* ***OC_UNOCCUPIED****.*

  To process the timer you will need to use a ***when(timer_expires(…))****.*

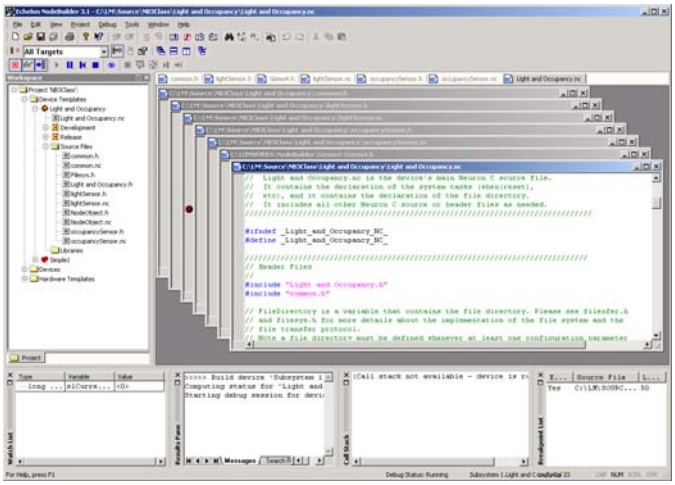
  1. Place variable declarations and constant definitions specific to this functional block at the end of *occupancySensor.h* file.
  2. Add code to the end of *occupancySensor.nc* file to process the push button input change.
  3. Note that **ioButton1** is active low.
  4. Add code, if necessary, to the **occupancySensorDirector(..)** function in the *occupancySensor.nc* file to define behavior during various states of the functional block. (e.g. when the device is being *Reset*, *Enabled*, *Disabled*, etc)
  5. If the **Load after Build** option is not already set in your project, go to the **Project** menu, select **Settings**, select the **Build** tab, check the **Load after Build** checkbox and click **OK**. Perform a build and load of the Light and Occupancy device by right-clicking the "Light and Occupancy" device shape in the Project pane and selecting **Build**.
  6. If there are problems during the build, use the editor to fix the code for both the Light Sensor and Occupancy Sensor functional blocks.

Part 5: Debug your code

NodeBuilder has a full-featured debugger built into the development environment, which includes facilities for setting breakpoints, stepping through and over code, and watching variables. In this section you will use the debugger to examine the behavior of the I/O processing implemented in the previous sections.

Important: *To perform any part of this section, you must have successfully built and loaded code onto your development hardware.*

1. To launch the debugger, switch back to the LonMaker drawing for this NodeBuilder project, right-click the device shape that represents the development device you just loaded and select **Debug**. This should push LonMaker to the background and bring the NodeBuilder Debug environment to the foreground.



   (In some versions of Windows you will have to switch to the NodeBuilder environment manually).

2. Go to the Project pane and double click the *lightSensor.nc* file to bring it to the front of the editor pane within the debugger. Maximize the *lightSensor.nc* window so that you can view more of the code.

3. Right click on the first line of code (not declarations) within your io_changes when clause in *lightSensor.nc* and select **Toggle Breakpoint**. The debugger will warn you that the code resides in Flash memory. Check the "Don't show this message" checkbox in this Warning screen and click **OK** to continue.

4. Turn the dial on the Gizmo 4, and look at *lightSensor.nc* to see if the breakpoint is caught.

When the debugger is paused or reaches a breakpoint you should find the encountered break point containing a yellow arrow, the Pause button on the Debugger toolbar will become grayed out, and both the Debugger Window and Watch List will contain relevant information as to the execution of the code.

5. As in other debuggers, the **Step Into** command (F11) will allow you to step through the code inside a function that is being called at the current statement awaiting execution. If you wish to execute the entire function in a single step then the debugger also provides the **Step Over** command (F10) for this purpose. If the current statement awaiting execution is not a function call then there is no difference between the **Step Into** and **Step Over** command.

To watch the values of variables during the course of execution you may add them to the Watch List.

6. Right-click on the variable you would like to Watch (it can be in either a ***.nc*** or a ***.h*** file), select **Watch Variable**, and click the **Add Watch** button in the dialog box.
7. Use breakpoints, step tools, and the watch list to verify operation of the I/O processing for both the *input quadrature* and *input bit* objects. When you have finished your debug session, go to the **Debug** menu, and select **Stop Debugging → Current Device**.

Part 6: Test your device through the device interface

Once you have verified that internal operation is performing as designed, NodeBuilder allows you to seamlessly step back to the view of the end user and test the behavior through the device interface. The LonMaker tool and its power as a network integration tool enable this transition.

1. Switch back to the LonMaker drawing, right-click the *Light Sensor* functional block and select **Browse**.
2. Click back on the LonMaker drawing, without closing the LonMaker Browser, right-click the *Occupancy Sensor* functional block, and select **Browse**. (An alternative to steps 1 and 2 would be to right-click on the device shape and select **Browse**).
3. With the LonMaker Browser in the foreground you should see your two network variables and their current values listed in the table. To enable constant polling for the devices, click the **Monitor All On** icon, or press Ctrl+B.
4. Turn the dial (shaft encoder) on the Gizmo 4 and verify that the increment speed and minimum and maximum values have all been implemented correctly.
5. Press the IO_6 push button and measure the time that it takes the value to switch from OC_OCCUPIED back to OC_UNOCCUPIED to determine if the pulse stretching timer was implemented correctly.
6. When you have finished the Device interface testing, close the LonMaker Browser.

Part 7: Optional - Add Gizmo 4 display and piezo support

In addition to providing some basic I/O objects, the Gizmo 4 has several more complicated I/O devices and a prototype area for your own custom hardware. A PIC microcontroller (connected to the Neuron Chip via an SPI interface also known as NeuroWire in Neuron C) drives the display and several library functions have been provided to enable easy access to this resource. A frequency I/O object available in the Neuron C language can drive a piezo buzzer, and for the Gizmo 4 piezo there is a set of library functions that will allow you to play various tones with simple code.

1. Add support for the piezo buzzer by using a define statement in *common.h* to set the USE_FREQUENCY option. This #define statement must occur in a line of code before the *gizmo4.h* #include so that the compiler will parse the Gizmo 4 library correctly.
2. Find the following functions in *gizmo4.h*, and read the usage guidelines in the header file to so that you can make calls to these functions in your code: `GizmoDisplayString(), GizmoPlayTune(), GizmoDisplayNumber(), GizmoDisplayFormattedNumber().`
3. Add variable declarations and constants for this functionality to the header file for the object that you intend to enhance with the display and piezo output.
4. Add code to *lightSensor.nc* and *occupancySensor.nc* to add the display and piezo functionality to the device.
5. Update the director functions in *lightSensor.nc* and *occupancySensor.nc* to manage the behavior of the display and piezo during the various functional block states (e.g. when the device goes to *Reset, Enabled, Disabled, etc*). This is particularly important for the display during the reset state, because you should make sure that you initialize the display by setting up a default message or displaying a blank screen.

Extra Notes

- **Locked out processing**

  In the solution for this lab you might notice that the `when` *clauses that process the I/O inputs, performs a check of*

  ```
  fblockNormalNotLockedOut(lightSensor::global_index)
  ```

  *The intent of this if statement is to prevent processing for the given object should it be disabled (fblockData[fblockIndex].**objectStatus.disabled**), it has failed self test(fblockData[fblockIndex].**objectStatus.failed_self_test**), or it has been locked out during initialization which has not been cleared by the director function (fblockData[fblockIndex].**objectStatus.failed_locked_out**). The locked-out flag is set for each object at reset and is then cleared once all initialization has occurred by the director function of the given object.*

- **Functional Block Member Names**

  It is recommended that if you wish to read or write to a network variable that you do so by referencing the functional block and the member name rather than using the NV directly.

  For example, in this laboratory to reference nvoLuxLevel you would use lightSensor::nvoLuxLevel.

Laboratory 5: Adding Optional Configuration Properties

Overview

Up to this point, only mandatory network variables have been used in our interoperable 'Light and Occupancy' device. However, the LONMARK Interoperability Guidelines *provide optional features that can be implemented as part of the standard device interface of a functional profile.*

In this lab we will incorporate two very useful optional configuration properties to the functional blocks implemented in the 'Light and Occupancy' device. These configuration properties are commonly referred to as Throttle and Heartbeat. Throttle is used to prevent a device from flooding the network with unnecessary traffic while Heartbeat is a network variable update that is automatically sent if the network variable has not otherwise been updated for a certain period of time, this behavior is used by the receiving as an indication that the sending device is still alive.

Objectives

- Implement optional configuration properties
- Identify how NodeBuilder manages the Program ID when the device interface changes
- Use Neuron C syntax for referencing network variable and configuration property members of a functional block

Description

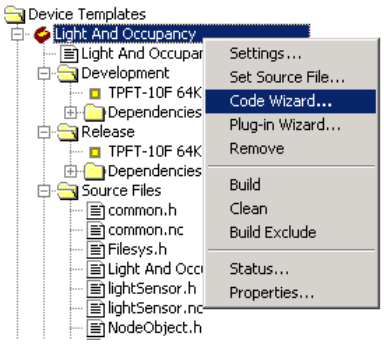
In your particular case, the Light sensor is a potential candidate to send constant updates of the current light level to the network, since light levels are constantly changing. You will avoid flooding the network by implementing the optional *nciMinSendT* configuration property available in the Light Sensor Functional Profile. *nciMinSendT* is defined as the minimum amount of time allowed between two consecutive updates of a network variable value to the network (*Throttle*).

You will configure the heartbeat by using optional configuration properties available in both the Light Sensor and the Occupancy Sensor Functional Profiles. Their names are nciMaxSendT and nciHeartBeat respectively (The names are different due to historical reasons within LonMark, however the behavior should be the same).

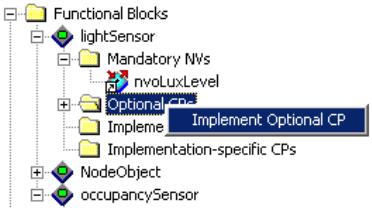
Instructions

Part 1: Add optional configuration properties with the NodeBuilder Code Wizard

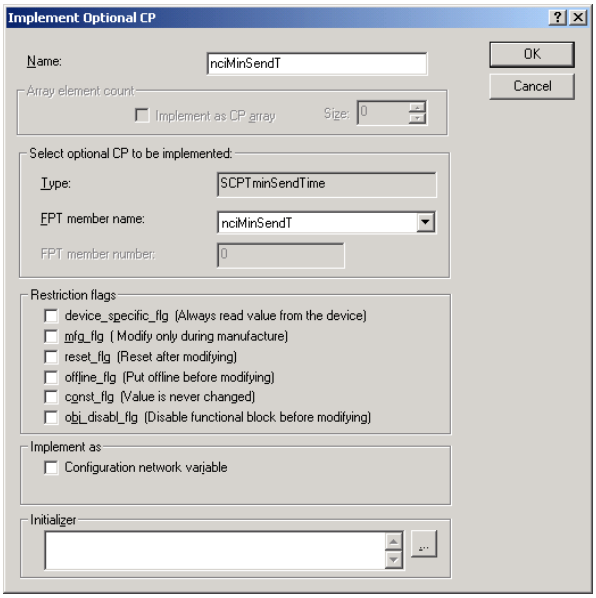
1. If you do not still have LonMaker and NodeBuilder running, open the LonMaker network from the Laboratory 4 and launch the NodeBuilder development tool.
2. In the Project pane, right click on the **Light and Occupancy** Device Template and select **Code Wizard** from the drop-down menu.



3. In the Program interface pane, expand the *lightSensor* functional block, right click on **Optional CPs**, and click on **Implement Optional CP**.



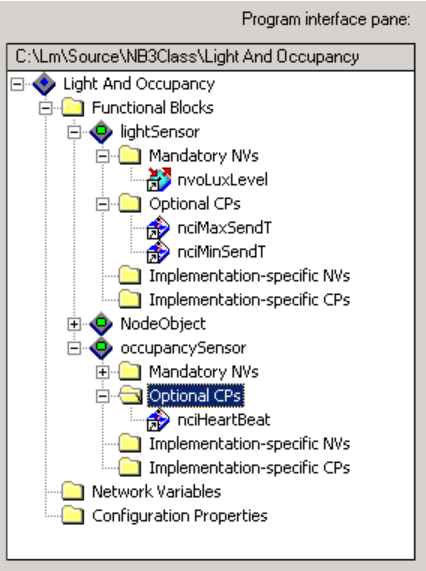
*The Code Wizard launches the **Implement Optional CP** dialog box, shown in the picture below.*



4. Select *nciMinSendT* in the **FPT member name** field, the **Name** field will automatically be updated.

The Code Wizard only lists those configuration properties that are defined as optional in the Light Sensor functional profile, ensuring that the correct configuration properties are used. nciMinSendT is the configuration property used for throttling the network variable updates generated by the Light Sensor.

5. Click **OK** and repeat steps 3 and 4 for adding *nciMaxSendT* to the lightSensor functional block and *nciHeartBeat* to the occupancySensor Functional Block.
6. The program interface pane should look like the one in the picture below. Click **Generate Code**, and the Code Wizard will update the source files with the new information.
7. Build the project by right-clicking on the *Light and Occupancy* device template on the Project pane and then select **Build** (The *LonMaker Application Image Wizard* window may come up, click Finish and when it is done switch back to *NodeBuilder*). Right click on the *Light and Occupancy* device template again and select **Settings**. Pay attention to the Program ID, the model number (last byte) is no longer 0. It has been increased to 1 by NodeBuilder.



Why was the model number increased by one?

_____

_____

_____

Part 2: Review the new code added by the Code Wizard

The following block of code was extracted from the *lightSensor.h* header file. The Code Wizard automatically added lines 2, 3, 9, 10, 11 and 12 when you added the optional configuration properties.

```
1   //<Fblock CP Family Declarations>
2   SCPTmaxSendTime cp_family nciMaxSendT;
3   SCPTminSendTime cp_family nciMinSendT;
4   //
5   //<Fblock Input NV Declarations>
6   //
7   //<Fblock Output NV Declarations>
8   network output SNVT_lux nvoLuxLevel
9   nv_properties {
```

```
10    nciMaxSendT,
11    nciMinSendT
12  };
```

Similarly, the block of code that follows was extracted from the *occupancySensor.h* header file. Lines 2, 17, 18 and 19 are new.

```
1   //<Fblock CP Family Declarations>
2   SCPTmaxSendTime cp_family nciHeartBeat;
3   //
4   //<Fblock Input NV Declarations>
5   //
6   //<Fblock Output NV Declarations>
7   network output SNVT_occupancy nvoOccup = OC_UNOCCUPIED;
8   //
9   //<Fblock Declaration>
10  fblock SFPToccupancySensor {
11    nvoOccup    implements nvoOccup;
12    director    occupancySensorDirector;
13  } occupancySensor
14  #ifdef USE_EXTERNAL_NAME
15  external_name("occupancySensor")
16  #endif
17  fb_properties {
18    nciHeartBeat
19  };
```

What is the difference between the configuration properties declarations in the two files?

_____

_____

_____

Part 3: Add Neuron C code to implement throttling and heartbeat

The value of nciMinSendT *determines the throttling time. When this value is 0, throttling is deactivated and network variable updates can be propagated as fast as the value changes.*

The values of nciMaxSendT *for the Light Sensor functional block and* nciHeartBeat *for the Occupancy Sensor functional block determine the maximum amount of time a network variable can be left without being updated. If the time elapses and the sensed condition does not indicate that the value of the network variable has changed, then a Heartbeat will be generated. The heartbeat updates the network variable with its current (same) value, effectively propagating the value over the network and serving as an "I am alive" signal for devices that are expecting this heartbeat. A value of 0 for the heartbeat means that this feature is deactivated and the network variable will only be updated when the sensed condition changes.*

In the next steps you will write the algorithm that implements throttling and heartbeat for the nvoLuxLevel *network variable, which is used by the Light Sensor functional block to send the current sensed value to the network. You will need to replace any instructions in <> brackets with functional code.*

Optionally you can implement everything yourself, in this case it would be easier to have an application timer for the throttle and one for the heartbeat and synchronize them accordingly.

Please take the time to understand the code you are entering rather than just copying it – you may need to use this mechanism in later labs. This code is an example of one way to implement throttle and heartbeat timers and not the only way it can be done. For example, you could use an application timer for the throttle and another for the heartbeat.

1. Edit the *lightSensor.h* file to add the following:

```
#define MAX_LUX 1000L
#define MIN_LUX 0L
signed long slLightSensorLux;
//Both Heartbeat and Throttle have a resolution of 100ms
#define LIGHT_SENSOR_TICK_CORE 100L
//A milliseconds counter will expire every 100ms
mtimer repeating LightSensorTickTimer = LIGHT_SENSOR_TICK_CORE;
//The next two counters will be processed within the timer event
unsigned long ulLightSensorThrottleCounter;  //counts up
unsigned long ulLightSensorHeartbeatCounter; //counts up
```

2. Edit the *lightSensor.nc* file as follows:

```
when (io_update_occurs(ioWheel))
{
   //Only process if the functional block is operating normally
   if (fblockNormalNotLockedOut(lightSensor::global_index))
   {      //slCurrentLuxLevel mantains the current lux level,
      //it has to be unsigned long becuase the input_value
      //can be negative.  (input_value is scaled up for responsivness)
      slCurrentLuxLevel += (input_value * 5);

      //Check the new value is within range
      slCurrentLuxLevel = max(MIN_LUX, slCurrentLuxLevel);
      slCurrentLuxLevel = min(MAX_LUX, slCurrentLuxLevel);
```

```
      //Can we transmit now or do we have to wait? (Throttle)
      // (note: cpMinSendT == 0 means throttle is disabled)
      if ((ulLightSensorThrottleCounter >=
           lightSensor::nvoLuxLevel::nciMinSendT)||
          (lightSensor::nvoLuxLevel::nciMinSendT == 0))
      {
         // Everything ok, so reset throttle and heartbeat timers
         <reset throttle counter to 0>
         <reset heartbeat counter to 0>

         // Transmit new value
         <assign slCurrentLuxLevel to lightSensor::nvoLuxLevel>
      }

      //Update LCD with the new value
      GizmoDisplayFormattedNumber(12, 2, slCurrentLuxLevel,
                                  10, 4, 0, ' ');
   }
}

when (timer_expires(LightSensorTickTimer))
{
   //Only process if the functional block is operating normally
   if (fblockNormalNotLockedOut(lightSensor::global_index))
   {
      //------ Manage throttle (only if it is enabled) -------------
      if (lightSensor::nvoLuxLevel::nciMinSendT != 0)
      {
         if (ulLightSensorThrottleCounter <
             lightSensor::nvoLuxLevel::nciMinSendT)
         {
            ++ulLightSensorThrottleCounter;
         }
         else
         {
            // Has a change been made to slCurrentLuxLevel,
            // yet we were throttled back?
            if (lightSensor::nvoLuxLevel !=
               (unsigned long)slCurrentLuxLevel)
            {
               <assign slCurrentLuxLevel to lightSensor::nvoLuxLevel>
               <reset throttle counter to 0>
               <reset heartbeat counter to 0>
            }
         }
```

```
      }
      //------ Manage heartbeats (only if they are enabled) -----------
      if (lightSensor::nvoLuxLevel::nciMaxSendT != 0)
      {
         if (ulLightSensorHeartbeatCounter <
             lightSensor::nvoLuxLevel::nciMaxSendT)
         {
            <increment heartbeat counter>
         }
         else
         {
            //Send heartbeat out to the network
            <assign slCurrentLuxLevel to lightSensor::nvoLuxLevel>
            <reset throttle counter to 0>
            <reset heartbeat counter to 0>
         }
      }
   }
}
```

3. Finally, add the following to the Light Sensor director function

   ```
   void lightSensorDirector (unsigned uFblockIndex, int iCommand)
   ```

   inside the FBC_WHEN_RESET section

   ```
   else if ((TFblock_command)iCommand == FBC_WHEN_RESET)
   ```

   ```
   //Initialize the LCD screen.
   GizmoClearDisplay();
   GizmoDisplayString(0,0,"Light Sensor");
   GizmoDisplayString(0,2,"Lux Level : 0");
   ```

4. Test the heartbeat and throttle functionality using the *NodeBuilder* debugger and *LonMaker Browser*. It may help to bind the device to another or to a host input NV and then use the LonScanner Protocol Analyzer, if it is installed on your PC, to observe the updates.
5. Build the device template and fix any errors you may encounter.

Note (using the LonMaker Browser) that by default both the heartbeat and throttle are enabled (their values are greater than 0). As part of your tests, set those values to 0 and make sure that the lightSensor is able to transmit immediately and that there are no heartbeats propagated. Then set the throttle to 500ms and the heartbeat to 5s and ensure that both function as expected.

Can you test both throttle and heartbeat functionality without having the lightSensor bound to another device? How?

Using all the documentation available (including the LonMark website), can you explain why the throttle and heartbeat have default values different from 0?

Can you test both throttle and heartbeat functionality without having the lightSensor bound to another device? How?

Can you test both throttle and heartbeat functionality without having the lightSensor bound to another device? How?

Can you test both throttle and heartbeat functionality without having the lightSensor bound to another device? How?

Can you test both throttle and heartbeat functionality without having the lightSensor bound to another device? How?

Laboratory 6: Creating Resource Files

Overview

There will be occasions when existing SNVTs, LONMARK SFPTs, or data formatting do not suit a particular application. In this instance a manufacturer must be able to create a device using the exact data types appropriate to a given application.

Objective

- Create new:
  - resource file sets
  - user network variable types
  - enumerations
  - configuration property types
  - functional profiles
  - formats

Description

You will use the NodeBuilder Resource Editor to add all the resources listed in the Objectives. You will have a chance to use the user types created in Laboratory 8.

Instructions

Part 1: Open the NodeBuilder Resource Editor

1. Open the NodeBuilder Resource Editor by selecting the **Echelon NodeBuilder Software** folder in your **Start** menu and selecting **NodeBuilder Resource Editor**.

For additional ways to access the Resource Editor, refer to the Using the Resource Editor *section in the* NodeBuilder User's Guide.

The Resource Editor can be used to create and modify user resource files, so that you can include them in your NodeBuilder project. The resource file catalog is a file with a .CAT extension that determines what directories will be searched for resource files. By default, this catalog file is contained in c:\lonworks\types directory.

Part 2: Add a new set of resource files to the catalog

1. Right-click the top level of the file tree in the NodeBuilder Resource Editor (i.e. the .CAT file) and select **New Resource File Set**.
2. Choose **Scope 3 - Manufacturer Class**.
3. Open the Program ID calculator by clicking the **Calculator** button. Specify "Examples" as the manufacturer. Since the scope is "Manufacturer", the other parts of the Program ID are not relevant for this set of resource files.
4. Click **OK** to close the Program ID calculator. The base name for the resource files is now "Examples".
5. Click on the **Header** tab and fill in the required information (creator, phone number, email address, and web address) with your own or made up information.
6. Click **OK** to close the **New Resource File Set** window. You should now see the c:\lonworks\types\user\examples directory in the catalog

Part 3: Add a new network variable type

We want to create a type called UNVT_alarm, which will be used in our user defined functional profile template. This type will be an enumeration with the following values:

NO_INTRUDER	*0*
INTRUDER	*1*
CODE1_WRONG	*2*
CODE2_WRONG	*3*
CODE3_WRONG	*4*

- **Create the Enumeration type**

  We need to create the enumeration first.

  1. Expand the *Examples* folder. It contains folders for each of the different types you can create: network variable types, configuration property types, functional profile templates, enumerations, resource strings, and formats.
  2. Right-click the *Enumeration* folder and select **New Enum**.
  3. In the **Tag Name** text box, type the enumeration name "alarm".
  4. Select a row in the table below
  5. Double-click the name of the first member and change it from Member1 to "NO_INTRUDER".
  6. Double-click anywhere on the **Name** column. This will add another member. Change the name of this member to "INTRUDER".
  7. Repeat the above process to add the rest of the five members of this enumeration.
  8. Click **OK** to save and close the enumeration window. The *alarm* enumeration type is now listed in the *Enumerations* folder.

- **Create the Network Variable type**

  You are now going to create the UNVT based on this enumeration.

  1. Right click the *Network Variable Type* folder and select **New NVT**.
  2. Enter "UNVT_alarm" in the **Name** field.
  3. In the **Data Type** frame, select "ENUMERATED" from the list.
  4. Fill in the enumeration information:
     - **Enum Scope** is 3
     - **Enum set** is alarm
  5. Select the **Strings** tab and enter "Alarm" for the **Name**, "Sends the alarm status" for the **Comment**.
  6. Click **OK** to close the **New NVT** window. The *UNVT_alarm* type now appears in the *Network Variable Types* list.

  Note that a new Format type is created automatically for you.

<u>Part 4: Add a new configuration property type</u>

1. Right-click the *Configuration Property Type* folder and select **New CPT**.
2. Enter "UCPTcode" in the **Name** field.

3. Select "REFERENCE" for the **Data Type**. This means that the configuration property type will be based on an existing network variable type.
4. Select the type "SNVT_count" in the list of standard resource files in the **NVT name** field.
5. In the **Referenced type range override and default value** frame at the bottom left of the window, select the "Default" value in the drop-down list, and set it to "1234" hex value in the text field.
6. Click **OK** to close the window.

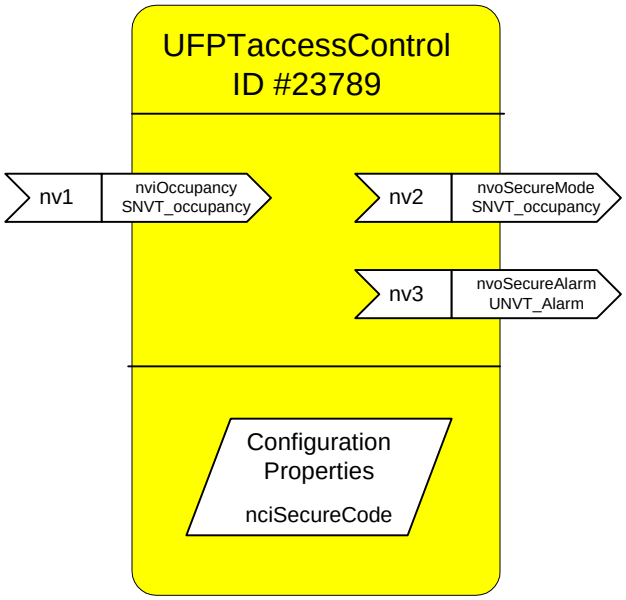
Note that a new Format type is created automatically for you.

Part 5: Add new functional profile templates

- **Define the new functional profile template**

1. Right-click the *Functional Profile Template* folder and select **New FPT**.
2. Change the name of the new FPT to "UFPTaccessControl".
3. Right-click *UFPTaccessControl* and select **Open**. On the right, you see the FPT general definition. Change the FPT key to "23789".

On the left, you see a tree that contains the standard resources files as well as our own "Examples" set of files. We want to create the following functional profile template. All the network variables are mandatory and there is one configuration property.



- **Add the network variables to the functional profile template**

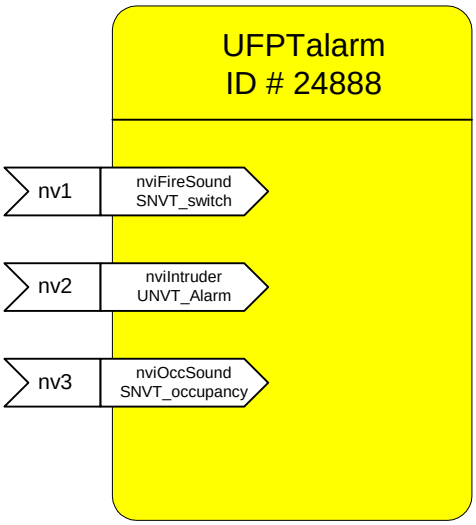
  1. Browse the resource files and find the type *SNVT_occupancy*.
  2. Drag and drop it into the *Mandatory NVs* section of the *UFPTaccessControl* folder. This will create a mandatory network variable in the FPT, called "nviManNV1" by default.
  3. Click on the network variable name. The network variable definition is displayed on the right. Change the **Name** field to "nviOccupancy".
  4. Repeat the above steps to create the two other network variables. You need to select the **Output** radio button in the network variable definition. Notice that the name automatically changes to start with nvo (for network variable output). Choose nvoSecureAlarm as the principal network variable by checking the **Principal NV** checkbox.

- **Add the configuration property to the functional profile template**

  1. Browse the resource files and find the *UCPTcode* type in the *Configuration Properties* section. Drag and drop it in the *Mandatory CPs* folder of the FPT.
  2. Change its name to "nciSecureCode". In the **String information** frame, enter "SecureCode" as **String** and "Value of the security code" as **Comment**.
  3. Click **OK** to close the window.

- **Add a second the functional profile template**

  1. Follow the above procedures to create a second functional profile called "UFPTalarm", with FPT key 24888, based on the definition below. Choose nviFireSound as the principal NV.



Laboratory 7: Device Development Course Project

Overview

One of the primary advantages of LONWORKS network technology is the ability of device manufacturers to develop devices in isolation using LONMARK Interoperability standards for subsequent integration with other manufacturers devices.

The course project is designed to demonstrate the seamless integration of devices developed by different manufacturers.

Objectives

- Develop and test LONWORKS devices to a published specification using NodeBuilder.

Description

In this course project, you will build, along with your laboratory partner, two of the nine devices required to build the smart building network described below. To ensure that all devices will be developed for the final integration, they will be allocated on a first come, first served basis by your instructor. Please read the device overview and let your instructor know which two devices you wish to develop.

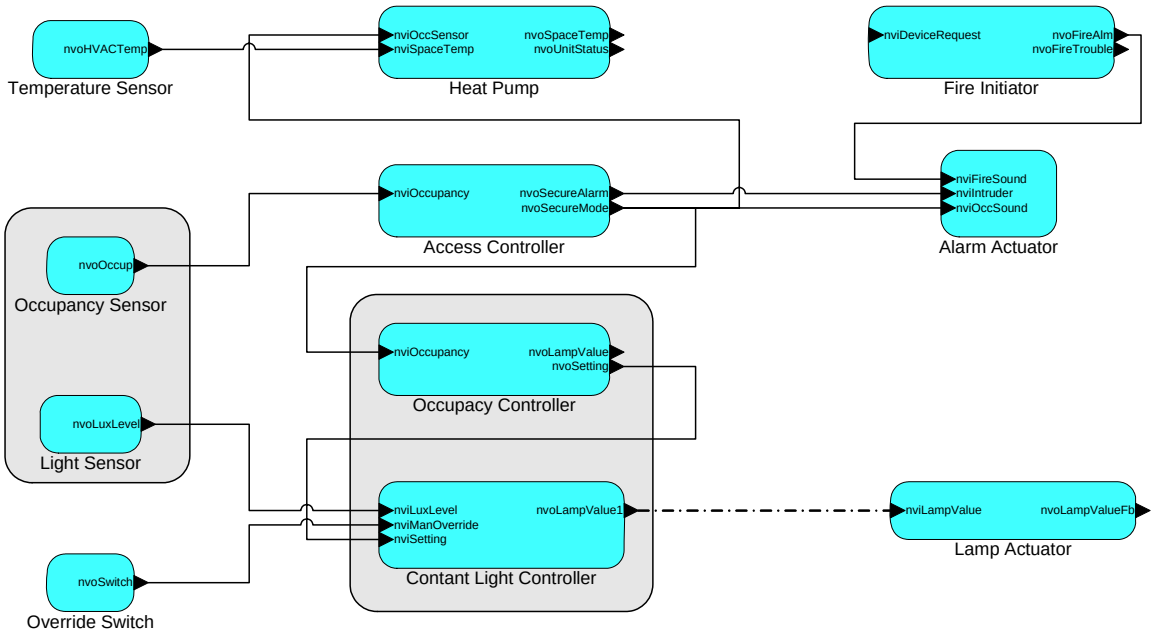
You previously developed the Light and Occupancy Sensor in Laboratories 3 through 5 and the resource files needed for the Light Occupancy Controller in Lab 7.

After development and testing of the devices in isolation you will integrate them together as a class using LonMaker in Laboratory 8.

Building Systems Specification

There are four building automation systems in this network: access control, heating, fire and lighting systems. When the user wants to gain access to the area, first the security code is input into the Access Control device. This enables the automatic lighting system and the heating system. At all times the area is protected by a fire detection and sounding system. When the area is unoccupied (but still unsecured), the lighting and ventilation systems save energy by turning off. Upon occupancy, these systems resume operation. When the area is secured again, with the Access Control device, the lighting and ventilation systems are disabled. Furthermore, when the area is secured, if an intrusion is detected, an alarm sounds. A functional description of each device in this network is given below. Please read these descriptions before continuing to the instructions.

The logical perspective of the smart building network is as follows:



Overview of Device Functional Descriptions

The devices are listed in their ascending level of complexity to allow you to pick the devices you feel most comfortable developing.

1. Override Light Switch

The lighting system is totally automatic; the lights are on if the access control system is in the 'unsecured state' and the area is occupied. However, for user convenience there is an override switch on the wall that allows users to force the lights on or off regardless or security or occupancy status. The override switch uses one of the Gizmo 4's push buttons for user input.

2. Fire Initiator

The Fire Initiator (fire detector) device uses a pushbutton on the attached Gizmo 4 to simulate a fire condition. The Alarm Actuator device needs the Fire Initiator information so that the area can be safely evacuated in case of fire.

3. Temperature Sensor

The Temperature Sensor device uses the attached Gizmo 4's temperature sensor to provide temperature readings of the area to the network. The temperature readings are used by the heat pump (described later) to determine whether to heat or cool the environment.

4. Lamp Actuator

The Lamp Actuator device receives information from the Constant Light Controller device and sets the lamp output to provide the specified amount of illumination. An LED on the Gizmo 4 is set to varying brightness to simulate the amount of illumination provided.

5. Alarm Indicator Device

The Alarm Indicator device audibly announces three separate conditions. The first condition indicates that there is a fire detected, using information from the Fire Initiator device. The second condition is an intrusion. This occurs when the Occupancy Sensor indicates that there is movement, and the Access Control device specifies that the area be in a 'secured' state. The third audible indication, a quick chirp, is generated whenever the Occupancy Controller indicates that there is movement. This is useful for an installer to be able to set the correct level of sensitivity for the Occupancy Sensor device.

As this device has user NV types, it uses the resource files previously created in lab 7.

6. Light and Occupancy Sensor (developed in laboratories 3-5)

The Light and Occupancy Sensor device uses the shaft encoder of the attached Gizmo 4 to simulate varying ambient lighting levels in the area, and one of the pushbuttons to simulate area occupancy. The Light Sensor information is needed by the Constant Light Controller (described later). The Access Control device, the Constant Light Controller device and the Heat Pump device need the Occupancy Sensor information.

7. Heat Pump

The Heat Pump receive ambient temperature information from the Temperature Sensor device and occupancy information from the access controller and decides whether to heat or cool the local area based on the difference between the current ambient temperature and the heat and cool preset thresholds. The Heat pump will only run when the building is occupied and unsecured.
The Heat Pump Device uses the Gimzo4s display to indicate heating or cooling and the piezzo to simulate the motor running.

8. Light and Occupancy Controller

The Constant Light Controller attempts to keep a constant light level in the area by using the information provided by the light sensor in the Light and Occupancy Sensor. As the ambient lighting level changes (usually due to changes in the amount of natural sunlight entering through the windows) the device's lamp output is set higher or lower.
The Occupancy Controller receives information from the Occupancy Sensor sends occupancy information to the network.

9. Access Control Device

The Access Control device provides a simple security system for the area. The Access Control device has a configurable four-digit PIN code. Before entering into the area, the occupant must first enter the PIN code so that the Access Control device can make the area 'unsecure'. The PIN code is entered using the attached Gizmo 4's quadrature shaft encoder and both of the push buttons. The Gizmo 4's display is used to display the current code choice. The Access Control device sends information to the Alarm Indicator device, the heat pump device and the Constant Light Controller device, and receives information from the Lighting and Occupancy Controller.

As this device has user NV types, it uses the resource files previously created in lab 7.

Instructions

Part 1: Choose your device

1. Make sure you have read the Device Functional Descriptions for the nine devices.
2. Let your instructor know your two choices, but remember it is on a first come first served basis. Continue only after the instructor has assigned all the devices to the class.

Part 2: Prepare for development

1. For the particular devices that you have chosen to implement, read the *Detailed Specification* and the associated Standard Functional Profile Definitions that you will find in your training bag for that device carefully.
2. Continue to use the LonMaker drawing and NodeBuilder project you have been using for all the previous labs.
3. Feel free to go back to previous laboratory exercises for help on how to implement your devices.

Part 3: Use NodeBuilder to develop your devices

1. Use default names for network variables and configuration properties as dictated by the relevant SFPT or UFPT.
2. All devices should use direct memory read write to implement configuration properties.
3. Ensure that any code you add to the code generated by the Code Wizard does no processing for a given functional block if the functional block is disabled.
4. Based all devices on the Gizmo 4/TPFT-10F Control Module combination (or the Gizmo 4/LTM-10A platform if that is your laboratory setup) and both the development and release hardware templates should be TPFT-10F 64K Control Module.
5. You may have to edit *gizmo4.h* and adjust the #defines USE_QUADRATURE and USE_FREQUENCY. Read the comments above the #defines for further explanation.

Part 4: Debug your device and test and integrate

1. Using a combination of the NodeBuilder debugger and the LonMaker Browser, verify that both the hardware and logical I/O are working correctly. Try to get your device to "break" by entering illegal values or setting it to an invalid state. Verify that your device does not malfunction. Make any Neuron C code changes as necessary.
2. After you have finished debugging your devices ask the instructor to test them.
3. Create a release version (using the same hardware) by creating a new device in the LonMaker tool using the release template. You will have to decommission the development device before commissioning the release device to avoid Neuron ID conflicts.
4. Your devices will be integrated with those from the rest of the class during Laboratory 8.
5. Find solutions for all the devices in the *solutions* folder on your student CD. However, please only look at the solutions if you are really stuck.

Detailed Device Specifications

Device 1: Override Switch

Description

The lighting system is totally automatic; the lights are on if the access control system is in the 'unsecured state' and the area is occupied. However, for user convenience there is an override switch on the wall that allows users to force the lights on or off regardless or security or occupancy status. To implement this function and comply with the LONMARK guidelines, the Override Light Switch has to implement two objects: a LONMARK Node Object, and a LONMARK Switch Functional Profile 3200.

Instructions

Create a development device in LonMaker called Override Switch and from within NodeBuilder's code wizard add a 3200 SFPT to the device. Do not use any optional network variables or configuration properties.

The program ID for the device should be: 9F:FF:FF:20:00:04:04:00.

The Switch Functional Block

Functional blocks can be enabled or disabled using a network management tool such as LonMaker. You should ensure that if the object is disabled that you do not process anything related to the object, either hardware or software. You can determine if the functional block is enabled or disabled by calling `fblockNormalNotLockedOut(<your functional block name>::global_index))` which will return true if the functional block is enabled (i.e. you can process data associated with it.

Whenever one of the pushbuttons on the Gizmo 4 hardware is pressed, the output NV should cycle through three states described in the following table.

NvoSwitch	State field	Value field
AUTO	0xFF	0xFF
ON	1	200
OFF	0	0

Sofware and Hardware Tips

You will only need to edit the files *common.h*, *switch1.h* & *switch1.nc*. Ensure that you have included the file *gizmo4.h* in *common.h*. Also ensure that if the object is disabled that you do not process any hardware inputs.

Use IO_7 push button (already declared as ioButton2 from within *gizmo4.h*).

Each press of the pushbutton advances through the cycle of output network variable values. When the device resets, the initial position should be AUTO.

Extra Credit Work

If you have time remaining, you could display the current state to the user on the Gizmo 4's display, and then implement a heartbeat using the optional configuration property.

Device 2: Fire Initiator

Description

The Fire Initiator device uses a pushbutton on the attached Gizmo 4 to simulate a fire condition. The Alarm Indicator device needs the Fire Initiator information so that the area can be safely evacuated in case of fire.

To implement these functions and comply with the LONMARK guidelines, the Fire Initiator must implement two objects: a LONMARK Node Object and a LONMARK Smoke (Intelligent) Fire Initiator Functional Profile 11002.

Instructions

Create a development device in LonMaker called Fire Initiator. Within the NodeBuilder Code Wizard, add an 11002 SFPT to the device. Do not use any optional network variables or configuration properties.

The program ID should be: 9F:FF:FF:6E:02:04:04:00.

Fire Initiator Functional Block

Functional blocks can be enabled or disabled using a network management tool such as LonMaker. You should ensure that if the object is disabled that you do not process anything related to the object, either hardware or software. You can determine if the functional block is enabled or disabled by calling
`fblockNormalNotLockedOut(<your functional block name>::global_index))`
which will return true if the functional block is enabled (i.e. you can process data associated with it).

For the purposes of this exercise the only nvoFireAlm need actually work, all other network variables and configuration properties can be ignored.

Software and Hardware Tips

You will need to edit *common.h*, *smokeFireInitiat.h* and *smokeFireInitiat.nc*.
Ensure that you have included the file *gizmo4.h* in *common.h*. Also ensure that if the object is disabled that you do not process any hardware inputs.

The Smoke (Intelligent) Fire Initiator Object uses the left (IO_7) pushbutton on the attached Gizmo 4 to simulate smoke detection.

Extra Credit Work

If you have any spare time you could flash “FIRE” on the Gizmo 4’s display when there is a fire condition and then implement a heartbeat using the optional configuration property.

Device 3: Temperature Sensor

Description

The Temperature Sensor device uses the attached Gizmo 4's temperature sensor to provide temperature readings of the room to the network. The temperature readings are needed by the heat pump device to determine whether to heat or cool the area.

To implement this function and comply with the LONMARK guidelines, the Temperature Sensor Device must implement two objects: a LONMARK Node Object, and a Temperature Sensor Functional Profile 1040.

Instructions

Create a development device in LonMaker called Temperature Sensor. Within the NodeBuilder Code Wizard, add a 1040 SFPT to the device. Do not use any optional network variables or configuration properties.

The program ID should be: 9F:FF:FF:0A:28:04:04:00.

The Temperature Sensor Functional Block

Ensure that if the object is disabled that you do not process anything related to the object.

By default the temperature sensor object has a single mandatory output network variable and two mandatory configuration properties, a heartbeat and a throttle. In addition to sending temperature using the single output network variable, implement both the heartbeat and throttle for the object.

Software and Hardware Tips

You will only need to edit the files common.h, hvacTempSPSensor.h & hvacTempSPSensor.nc.

Ensure that you have included the file *gizmo4.h* in *common.h*. Also, if the object is disabled, do not process any hardware inputs.

Refer to the file c:\LonWorks\NB\Gizmo4\gizmo4.h to see how to measure temperature using GizmoReadTemperature(). You should aim to read the temperature every 0.5 seconds.

Extra Credit Work

If you have time, display the temperature on the Gizmo 4's display using GizmoDisplayString().

Device 4: Lamp Actuator

Description

The lamp actuator turns on or off the lamp, or changes its luminous intensity depending on the output of the Lighting Controller located in the Light and Occupancy Controller device.

The device uses the Gizmo 4's LED connected to IO_1 to simulate the lamp, but does not use the file *gizmo4.h*.

To implement this function and comply with the LONMARK guidelines, the Lamp Actuator Device has to implement two objects: a LONMARK Node Object, and a Lamp Actuator Functional Profile 3040.

Instructions

Create a development device in LonMaker called LampActuator. Within the NodeBuilder Code Wizard, add a 3040 SFPT to the device. Include the optional configuration property nciDefOutput.

The program ID should be: 9F:FF:FF:1E:28:04:04:00.

The Lamp Actuator Functional Block

Functional blocks can be enabled or disabled using a network management tool such as LonMaker. Ensure that if the object is disabled that you do not process anything related to the object, either hardware or software. You can determine if the functional block is enabled or disabled by calling
`fblockNormalNotLockedOut(<your functional block name>::global_index))`
which will return true if the functional block is enabled (i.e. you can process data associated with it).

When the device powers up, the lamp shall go to the level specified by the optional configuration network variable, nciDefaultOutput.

When an update is received on the input network variable, the lamp will adjust accordingly and the feedback output network variable will reflect the new value on the input network variable.

Software and Hardware Tips

You will need to edit the files *lampActuator.h*, *lampActuator.nc*.

Use a pulse width I/O model to drive the LED and to avoid having to scale the output value. You can take the value field of the input network variable as a direct argument to the I/O model.

Device 5: Alarm Indicator Device

Description

The Alarm Indicator Device audibly announces three separate conditions. The first condition indicates that there is a fire detected (FIRE ALARM), using information from the Fire Initiator device. The second condition is an intrusion (INTRUSION ALERT). This occurs when the Occupancy Controller Object indicates that there is movement, and the Access Control device specifies that the area be in a 'secured' state. The third audible indication, a quick chirp, is generated whenever the Occupancy Controller indicates that there is movement. This one is useful for an installer to be able to set the correct level of sensitivity for the Occupancy Sensor device.

To implement these functions the Alarm Actuator must implement two objects: a LONMARK Node Object, and UFPT alarm (as previously defined in Laboratory 7).

Instructions

Create a development device in LonMaker called AlarmActuator. Within the NodeBuilder Code Wizard, add the UFPTalarm to the device.

The program ID should be: 9F:FF:FF:6E:06:06:04:01.

The Alarm Indicator Controller Functional Block

Functional blocks can be enabled or disabled using a network management tool such as LonMaker. You should ensure that if the object is disabled that you do not process anything related to the object, either hardware or software. You can determine if the functional block is enabled or disabled by calling `fblockNormalNotLockedOut(<your functional block name>::global_index))` which will return true if the functional block is enabled (i.e. you can process data associated with it).

The fire sound has the highest priority, and sounds until the fire condition is eliminated The intrusion sound has the next highest priority and sounds as long as there is intrusion indicated by the Access Control Device. If while sounding the intrusion alarm, a fire condition occurs, the fire sound starts immediately because fire has the highest priority. If there is neither a fire nor intrusion, the occupancy chirp is generated when occupancy starts in the area.

Tip: Once you have added the alarm function block to your device in the code wizard, rename it to *myalarm* to avoid recursive inclusion of the *alarm.h* file created in the resource file lab that you will have to include into your code.

Software Tips & Hardware Tips

You will only need to edit *common.h*, *alarm.h* and *alarm.nc*.

Use the function GizmoPlayTune() to make the required sounds. Experiment to select the appropriate noises. Instructions on how to use GizmoPlayTune() are found in c:\Lonworks\NB\Gizmo4\gizmo4.h.

You will need to add a path to the resource files, or save the files where the compiler will find them.

Device 6: Light and Occupancy Sensor

Description

This is the device you developed in laboratories 3 through 5.

Device 7: Heat Pump

Description

The heat pump receives ambient temperature from the temperature sensor device and compares this with configured thresholds to drive a pump and cooling/heating accordingly.

To implement this function and comply with the LONMARK guidelines, the Heat Pump Device must implement two objects: a LONMARK Node Object, and a Heat Pump SFPT 8503.

Instructions

Create a development device in LonMaker called Heat Pump. Within the NodeBuilder Code Wizard, add a SFPT 8503. Add only the optional configuration property, nciRcvHrtBt.

The program ID should be: 9F:FF:FF:50:32:04:15:00.

Heat Pump Functional Block

Functional blocks can be enabled or disabled using a network management tool such as LonMaker. You should ensure that if the object is disabled that you do not process anything related to the object, either hardware or software. You can determine if the functional block is enabled or disabled by calling

```
fblockNormalNotLockedOut(<your functional block name>::global_index))
```

which will return true if the functional block is enabled (i.e. you can process data associated with it).

This object is implemented using the optional network variable, nviOccSens, to ensure that the area is only heated when necessary.

Provided that occupancy is detected, when an update to nviSpaceTemp is made, the new value is compared with both nciSetpnts.occupied_heat and nciSetpnts.occupied_cool. If the value is within the two limits, the motor is stopped and neither heating or cooling is enabled (idle state). However, if it is below nciSetpnts.occupied_heat then the motor should run with heating enabled (heating state), or if it is above nciSetPnts.occupied_cool then the motor should run with cooling enabled (cooling state).

If heating or cooling is enabled and the motor running, and occupancy is no longer detected, then heating or cooling should be disabled and the motor stopped (idle state).

The configuration property, nciSetpnts, should be initialized per the SCPT definition:

Default (Values in Degrees C)	
occupied_cool	23
standby_cool	25
unoccupied_cool	28
occupied_heat	21
standby_heat	19
unoccupied_heat	16

How can you achieve this? Do you need to set an initial value in your code?

Simulate the heating, cooling and idle states by writing either “heating”,“cooling” or “idle” to the Gizmo 4’s display. When neither heating nor cooling is enabled and the motor is stopped, display “idle”.

Simulate the motor using the Gizmo 4’s piezo speaker.

Use the configuration property, nciRcvHrtBt, to determine a heartbeat monitoring window. For example, if an update from the ambient temperature sensor has not been received for the nciRcvHrtBt period, the heat pump should adopt a default state. In this case, the default state will be to go to the idle state and display “heartbeat failure”.

Although they are declared as part of the functional block’s logical interface, ignore nvoSpaceTemp and nvoUnitStatus for the purposes of this exercise.

The initial value of nviRcvHrtBt should be 0s. Again, how can you achieve this? Do you need to set an initial value in your code?

Software and Hardware Tips

You will only need to edit *common.h*, *sccHeatPump.h* and *sccHeatPump.nc*.

To simulate the motor, use GizmoPlayTune() with two notes and an endless loop. Adjust the length of the notes and their frequency to taste.

To best implement the receive heartbeat failure mechanism, use an application timer that is refreshed to the nciRcvHrtBt period on receipt of an update to nviSpaceTemp. Don’t forget that an nciRcvHrtBt value of zero disables this feature. Typically the receive heartbeat period is set to five times the expected update rate.

Extra Credit Work

If you have extra time available you could implement the heartbeat function applied to the primary output network variable.

Device 8: Lighting and Occupancy Controller Device

Description

In this device, the Constant Light Controller attempts to keep a constant light level in the area by using the information provided by the light sensor in the Light and Occupancy Sensor. As the ambient lighting level changes (usually due to changes in the amount of natural sunlight entering through the windows), the device's lamp output is set higher or lower.

The Occupancy Controller receives information from the Occupancy Sensor and uses this information to send occupancy information back to the network.

To implement these functions and comply with the LONMARK guidelines, the Lighting and Occupancy Controller Device must implement three objects: a LONMARK Node Object, a LONMARK Occupancy Controller Functional Profile 3071, and a LONMARK Constant Light Controller Functional Profile 3050.

Instructions

Create a development device in LonMaker called Lighting and Occupancy Controller. Within the NodeBuilder Code Wizard, add a SFPT 3050 and a SFPT 3071. Add the optional network variable, nvoSetting, to the Occupancy Controller object and add nviManualOverride to the light controller.

The program ID should be: 9F:FF:FF:1E:32:04:04:00.

The Occupancy Controller Functional Block

Functional blocks can be enabled or disabled using a network management tool such as LonMaker. You should ensure that if the object is disabled that you do not process anything related to the object, either hardware or software. You can determine if the functional block is enabled or disabled by calling

```
fblockNormalNotLockedOut(<your functional block name>::global_index))
```

which will return true if the functional block is enabled (i.e. you can process data associated with it).

Ensure that you have included the file *gizmo4.h* in *common.h*. Also ensure that if the object is disabled, you do not process any hardware inputs.

For this simplified implementation:

If nviOccupancy is set to OC_OCCUPIED, then nvoSetting.function should be set to SET_ON, with nvoSetting.setting and nvoSetting.rotation set to 0. If nviOccupancy is set to OC_UNOCCUPIED, then nvoSetting.function should be set to SET_OFF, with both nvoSetting.setting and nvoSetting.rotation set to 0,

Occupancy Controller Software and Hardware Tips

The Occupancy Controller does not require any I/O hardware to function.

The Constant Light Controller Functional Block

Functional blocks can be enabled or disabled using a network management tool such as LonMaker. You should ensure that if the object is disabled that you do not process anything related to the object, either hardware or software. You can determine if the functional block is enabled or disabled by calling

```
fblockNormalNotLockedOut(<your functional block name>::global_index))
```

which will return true if the functional block is enabled (i.e. you can process data associated with it).

The configuration property, nciLuxSetpoint, should be initialized to 340 lux. How might you achieve this?

The light controller should function as follows:

If the function field of nviSetting is set to SET_ON:

> If nviManOverride is set to auto (0xff,0xff), then the constant light controller should compare the current ambient light level presented at nviLuxLevel against the setpoint CP nciLuxSetpoint and drive nvoLampLevel accordingly by ramping to the target value over a suitable period of time.
>
> If nviManOverride is set to 0,0, then nvoLampLevel should be set to 0,0 and the lamp turned off.
>
> If nviManOverride is set to 100,1, then nvoLampLevel should be set to 100,1 and the lamp turned full on.

If the setting filed of nviSetting is set to SET_OFF:

> Then nvoLampLevel should be set to 0,0 and the lamp turned off.

Constant Light Controller Software and Hardware Tips

The Lighting Controller device does not require any I/O hardware to function

Device 9: Access Controller Device

Description

The access controller is used to determine the security level of the building and raise alarms as necessary. It also provides access information to the heating and lighting systems.

Using the push buttons of the Gizmo 4 and its shaft encoder, the user can enter a key code to allow access to the building (unsecure mode) or deny access to the building (secure mode).

To implement these functions the Access Controller must implement two objects: a LONMARK Node Object, and UFPT access controller (as previously defined in Laboratory 7).

Instructions

Create a development device in LonMaker called Access Controller. Within the NodeBuilder Code Wizard, add the UFPTaccessController to the device.

The program ID should be: 9F:FF:FF:32:00:05:04:01.

The Access Controller Functional Block

Functional blocks can be enabled or disabled using a network management tool such as LonMaker. You should ensure that if the object is disabled that you do not process anything related to the object, either hardware or software. You can determine if the functional block is enabled or disabled by calling

```
fblockNormalNotLockedOut(<your functional block name>::global_index))
```

which will return true if the functional block is enabled (i.e. you can process data associated with it).

The Access Controller should have the following controls, indicators, and events:

When the Access Controller starts, it is in a 'secure' state and prompts the user to enter their PIN number on the Gizmo 4's display. When in the 'secure' state, if occupancy is detected on nviOccupancy, an alarm event is initiated by setting the output network variable nvoSecureAlarm to INTRUDER.

The only way to get out of the 'secure' state is to input the secret PIN code. The user enters the code by turning the shaft on the attached Gizmo 4 hardware. The shaft encoder detects movement either clockwise or counter-clockwise. The Neuron C application reads the shaft encoder and displays a one-digit number on the Gizmo 4's display. The range of displayed is '0' through '9'. When the user sees the correct digit of the secret code, he presses the 'NEXT DIGIT' (left) pushbutton to advance and enter the next digit. After keying in all 4 digits of the PIN number, the user presses the OK (right) push button, and the application verifies the entered PIN number. If the entered number matches the one stored in the device, the display is updated accordingly and the Access Controller enters the 'insecure' state.

The user has three attempts to enter the correct code. At each failed attempt, nvoSecureAlarm is set first to CODE1_WRONG, then to CODE2_WRONG, and finally to CODE3_WRONG. After the third failed attempt, the user is no longer be able to enter any further attempts until the device is reset. The display reflects the status of each attempt.

When in the secure state, the output network variable, nvoSecureMode, is not changed regardless of updates to the input network variable nviOccupancy, since equipment that is enabled from the output, such as the heat pump or constant light controller, only need to be enabled when occupancy is expected and should not provide intruders with a comfortable environment to work in.

To re-enter the 'secured' state, the user presses the right pushbutton on the Gizmo 4. When entering the 'secured' state, the display is updated accordingly.

To implement this function, the Access Controller device must implement two objects: a LONMARK Node Object, and an Access Contoller Object (using the previously created UPFT named UFPTaccessControl). You will need to move the resource file to where this project can find it – the Resource Editor places it in a standard directory – or include the path.

Software and Hardware tips

The shaft encoder is used with the quadrature I/O device driver. On the Gizmo 4 device, one 'click' of the detect on the shaft yields a numerical difference of 2. Therefore, when displaying the desired security code digit '0' to '9', remember that you may need to divide the quadrature input by 2. You may need to experiment. You will need to determine what the secret code is. How should you configure this code?

Laboratory 8: Device Integration

Overview

Devices developed by different manufacturers can be integrated together using a network management tool such a LonMaker without custom software or coordination between the manufacturers. LONMARK Standard Functional Profiles offer plug-and-play interoperability between devices from different manufacturers.

Objectives

- Integrate devices developed in isolation using the LonMaker tool
- Test the network using the LonMaker Browser and LonMaker monitoring tools
- Test the network with the LonScanner Protocol Analyser

Instructions

Your instructor will pass around a network cable to connect all the devices together. Connect your two devices to the cable.

Your instructor will request that one of you copy the format files you created in Laboratory 6 to a floppy disk so that they can be installed on the instructor's machine. The instructor will:

- Create the folder c:\LonWorks\Types\User\Examples (in this instance) on the target machine
- Copy the files into the folder
- Run the resource editor and add a new folder with a scope of 3

This process is normally automated at the time the relevant plug-in is installed.

Your instructor will use a LonMaker database previously created in engineered mode. When prompted by the instructor, press the service pin on the appropriate device so that it can be commissioned.

Your instructor will then talk you through the process of testing and debugging the network to ensure that it functions as designed. This will include testing the following:

- Access control system
- Heating System
- Fire System
- Lighting System
- Enabling of receive heartbeats on the Heat Pump device with a communications failure
- Configuring the Temperature Sensor with the plug-in from Laboratory 5
- Disabled function blocks